

Improving Computational Performance of Bias-Reduced Generalised Linear Models

Final Report • Third Year Project (CS350)

ABSTRACT

The `brglm2` R package implements bias-reduced estimation for generalised linear models (GLMs), addressing finite-sample bias in maximum likelihood through adjusted score equations (Kosmidis 2023). With approximately one million downloads annually, it is widely adopted across statistics, medicine, and the social sciences. The current quasi-Fisher scoring algorithm incurs $\mathcal{O}(np^2 + p^3)$ work per iteration through repeated QR decomposition of the weighted design matrix, rendering the package impractical at scale — precisely where bias-reduced inference is most needed.

This project develops a trust region framework with a preconditioned CG-Steihaug subproblem solver as a principled replacement for quasi-Fisher scoring within `brglm2`. The core fitting routine was refactored into a modular `compute_fit` helper enabling lazy evaluation of expensive operations. Profiling confirmed that hat value computation consumes over 85% of iteration time; this was addressed through adaptive QR scheduling and, for sparse inputs, a Cholesky-based hat value path with slot-level row scaling. The median bias-reduction adjustment was re-derived in closed matrix form, reducing per-iteration cost from $\mathcal{O}(np^3)$ to $\mathcal{O}(np^2)$.

On dense problems under `AS_median`, the combined implementation achieves a $31.65\times$ median speedup at $n = 1000$, $p = 200$ relative to the original. Sparse configurations show more dramatic gains: a high-dimensional one-hot scenario ($n = 10,000$, $p = 496$) completes in under a minute compared to approximately two hours under the original. Under `AS_mixed` on small datasets the trust region introduces modest overhead, as expected. Coefficient agreement with the original implementation is within 10^{-6} across all scenarios, and 392 of 394 existing tests pass. The implementation is backward-compatible and structured for upstream contribution to `brglm2` under GPL-3.

KEYWORDS

bias reduction

generalised linear models

trust region

CG-Steihaug

hat values

sparse matrices

R

brglm2

Oliver Clark

u5541018

Department of Statistics • University of Warwick

Supervised by **Prof. Ioannis Kosmidis**

Contents

CHAPTERS 1–3

1	Introduction	1
1.1	Motivation	1
1.2	Aims and Objectives	1
1.3	Report Structure	2
2	Background	2
2.1	Generalised Linear Models	2
2.2	Bias in Maximum Likelihood	3
2.3	Bias Reduction Methods	3
2.3.1	Mean Bias Reduction	
2.3.2	Median Bias Reduction	
2.3.3	Mixed Bias Reduction	
2.3.4	Maximum Penalised Likelihood	
2.3.5	Asymptotic Bias Correction	
2.3.6	Computational Similarities	
2.4	The brglm2 Computational Bottleneck	5
2.5	Trust Region Optimisation	6
2.5.1	Why Not Quasi-Fisher Scoring?	
2.5.2	The Trust Region Subproblem	
2.5.3	Subproblem Solver: CG-Steihaug	
3	Design & Implementation	8
3.1	Development Environment & Workflow	8
3.2	Codebase Architecture & Refactoring	9
3.2.1	The <code>compute_fit</code> Helper	
3.2.2	Boolean Flags for Lazy Evaluation	
3.3	Vectorisation of Median Adjustment	9
3.4	Trust Region Framework	11
3.4.1	Objective and Gradient	
3.4.2	Outer Loop and Radius Adaptation	
3.4.3	CG-Steihaug Subproblem Solver	
3.4.4	Jacobi Preconditioning	
3.4.5	Adaptive Inner Tolerance: Eisenstat-Walker	
3.4.6	Convergence Criterion	
3.5	Dispersion Estimation	13
3.5.1	Newton Step-Halving	
3.5.2	Analytical Geometry Injection	
3.6	Hat Value Computation	14
3.6.1	Exact Updates via QR Decomposition	
3.6.2	Adaptive Scheduling Strategy	
3.6.3	Rejected Alternatives	
3.7	Sparse Matrix Support	15
3.7.1	Cholesky Factorisation for Hat Values	
3.8	Backward Compatibility	16
3.9	Algorithm Summary	17

CHAPTERS 4–6 & APPENDICES

4	Testing & Validation	19
4.1	Benchmarking Infrastructure	19
4.2	Correctness Validation	20
4.3	Performance Benchmarks	21
4.3.1	Standard Package Datasets: <i>AS_mixed</i>	
4.3.2	Synthetic Dense Scenarios: <i>AS_mixed</i>	
4.3.3	Synthetic Dense Scenarios: <i>AS_median</i>	
4.3.4	Synthetic Sparse Scenarios: <i>AS_mixed</i>	
4.3.5	Synthetic Sparse Scenarios: <i>AS_median</i>	
5	Evaluation	26
5.1	Against Project Objectives	26
5.2	MoSCoW Requirements	27
5.3	Computational Performance	27
5.4	Limitations	28
6	Conclusions & Future Work	28
6.1	Summary of Aims	28
6.2	Achievements	29
6.3	Discussion	29
6.4	Future Work	29
6.5	Closing Statement	30
7	Legal, Social, Ethical & Professional	30
7.1	Ethical Considerations	30
7.2	Open Source Licensing	30
7.3	Professional Considerations	30
7.4	Social Impact	31
References		
Acknowledgements		
Appendices		
A	Profiling Output	
B	Benchmark Tables	

1 Introduction

This chapter establishes the context and motivation for the project, states the objectives, and signposts the structure of the report for the reader.

1.1 Motivation

Statistical models never enjoy the luxury of infinite data. When the number of observations is small relative to the number of parameters, the maximum likelihood estimator (MLE) is subject to systematic finite-sample bias of order $\mathcal{O}(n^{-1})$ (Firth 1993). In logistic regression this bias is particularly consequential: complete or quasi-complete separation of the response classes renders the MLE infinite, producing estimates and standard errors that are computationally and inferentially meaningless (Kosmidis and Firth 2009).

The `brglm2` R package (Kosmidis 2023) addresses these problems by replacing standard score equations with adjusted alternatives that reduce or eliminate finite-sample bias. The approach is theoretically principled, applicable across the exponential family, and implemented within R’s standard `glm()` interface. With approximately one million unique downloads annually from RStudio servers alone, `brglm2` has established itself as a widely-used tool across statistics, medicine, epidemiology, and the social sciences where often the usage is based out of a necessity for its properties rather than a practical nicety.

Despite its methodological strengths, the current implementation carries a significant computational cost. Every iteration of the quasi-Fisher scoring algorithm requires a full QR decomposition of the $n \times p$ weighted design matrix, incurring $\mathcal{O}(np^2 + p^3)$ work per step. Detailed profiling, described in Section 2.4, confirms that over 85% of total execution time is consumed by these linear-algebraic operations, a bottleneck that renders the package impractical for large n or high-dimensional p , precisely the settings where bias-reduced inference is most needed.

1.2 Aims and Objectives

This project aims to overcome these computational limitations by developing and implementing more efficient estimation algorithms within `brglm2`. The primary objectives are to:

1. Analyse and profile the existing implementation to identify and quantify computational bottlenecks.
2. Refactor core components of the `brglm2` codebase to improve modularity and efficiency.
3. Design and implement a trust region solver with a matrix-free CG-Steihaug subproblem solver, integrated with `brglm2`’s bias-reduction adjustment terms.
4. Develop an efficient hat value computation strategy that avoids redundant QR decompositions.
5. Integrate sparse matrix support for high-dimensional covariate structures.
6. Rigorously benchmark performance improvements against the original implementation.
7. Contribute production-ready code to `brglm2` under GPL-3.

1.3 Report Structure

Chapter 2 develops the theoretical foundations: GLM estimation, finite-sample bias, the five estimation methods in `brglm2`, the identified computational bottleneck, and the trust region framework chosen to address it. Chapter 3 describes the technical contributions in detail, from the modular refactoring of `brglmFit()` through to the complete trust region implementation with adaptive hat value scheduling and sparse matrix support. Chapter 4 presents the benchmarking infrastructure, correctness validation, and simulation study design. Chapter 5 evaluates outcomes against project objectives. Chapter 6 offers conclusions and directions for future work.

2 Background

This chapter develops the theoretical foundations required to understand both the statistical problem and the computational solution. It covers GLM estimation, sources of finite-sample bias, the estimation methods in `brglm2`, and the trust region framework adopted in this project.

2.1 Generalised Linear Models

Standard linear regression assumes a continuous, normally distributed response and models its mean as a linear combination of covariates. Many responses of practical interest violate this: binary outcomes, counts, and proportions require distributions other than Gaussian. Generalised linear models (GLMs) (Agresti 2015) extend linear regression to this broader class by connecting the expected response to the linear predictor through a link function, rather than assuming linearity of the mean itself.

A GLM has three components. The random component specifies that the observations $y_1, \dots, y_n$ are drawn independently from an exponential family distribution, a broad class that includes the Gaussian, binomial, Poisson, Gamma, and inverse Gaussian among others. The systematic component assembles a linear predictor $\eta_i = \mathbf{x}_i^T \beta$ from the p -dimensional covariate vector $\mathbf{x}_i$ and the unknown parameter vector β . The link function $g(\cdot)$ connects these: it maps the mean $\mu_i = \mathbb{E}[Y_i]$ to the linear predictor, $g(\mu_i) = \eta_i$.

GLM specification (Kosmidis, Kenne Pagui, and Sartori 2020).

Let $y_1, \dots, y_n$ be observations on independently distributed random variables $Y_1, \dots, Y_n$ with probability density functions taking the form of an exponential dispersion model (EDM)

$$f_{Y_i}(y; \theta_i, \phi) = \exp \left\{ \frac{y\theta_i - b(\theta_i) - c_1(y)}{\phi/m_i} - \frac{1}{2}a\left(-\frac{m_i}{\phi}\right) + c_2(y) \right\}$$

where $a(\cdot), b(\cdot), c_1(\cdot), c_2(\cdot)$ are sufficiently smooth with fixed observation weights $m_1, \dots, m_n$. θ_i is the natural parameter linked to μ_i via $b'(\theta_i) = \mu_i$, $b(\cdot)$ is the cumulant function whose second derivative gives the variance $\text{Var}(Y_i) = b''(\theta_i) a(\phi)$, and $\phi > 0$ is the dispersion parameter (fixed at 1 for binomial and Poisson).

The unknown β is estimated by maximising the log-likelihood, which for the exponential family is a concave function and can be found iteratively. The standard algorithm is Fisher scoring, equivalent to iteratively reweighted least squares (IRWLS). At each step, a weighted linear regression problem is solved using the current estimates. The update is:

$$\beta^{(k+1)} = \beta^{(k)} + (X^T W X)^{-1} X^T (\mathbf{y} - \mu)$$

where $W = \text{diag}(w_1, \dots, w_n)$ with observation weights $w_i = (d\mu_i/d\eta_i)^2 / \text{Var}(Y_i)$. The matrix $X^T W X$ is the

Fisher information which summarises the curvature of the log-likelihood and must be formed ($\mathcal{O}(np^2)$) and solved against the right-hand side ($\mathcal{O}(p^3)$, via QR or Cholesky factorisation) at every iteration, giving a total cost of $\mathcal{O}(np^2 + p^3)$ per step.

2.2 Bias in Maximum Likelihood Estimation

The MLE $\hat{\beta}$ is consistent and asymptotically normal, but in finite samples it is not centred at the truth. To see why, consider expanding the score equation around the true β , we note that the curvature terms that are negligible asymptotically become relevant when n is not large. Cordeiro and McCullagh (1991) and Kosmidis and Firth (2009) showed that for GLMs the expected value of the MLE satisfies:

$$\mathbb{E}[\hat{\beta}] = \beta + b(\beta) + \mathcal{O}(n^{-2})$$

where $b(\beta) = \mathcal{O}(n^{-1})$ is the leading bias term motivating this project. It depends on the first and second derivatives of the link function and the variance function, and does not vanish until $n \rightarrow \infty$. The $\mathcal{O}(n^{-2})$ remainder is negligible relative to b for all but very large samples.

The practical consequences of this bias are significant. In logistic regression, complete separation, where a linear combination of covariates perfectly predicts the outcome, causes the MLE to diverge to infinity (Kosmidis and Firth 2009). Even without separation, finite-sample bias inflates coefficient estimates away from zero, narrows confidence intervals incorrectly, and distorts hypothesis tests. These problems are most acute when p is non-negligible relative to n , precisely the regime where `brglm2` is most widely used.

2.3 Bias Reduction and Penalised Likelihood Methods

The five estimation approaches in `brglm2` each modify the standard score equations through a shared mechanism, differing only in how the adjustment term is formed; this section describes each method and identifies the computational bottleneck they share.

We start by introducing the usual score functions and information matrix for β and ϕ that is our foundation to applying any type of bias reduction. We keep this in the common language of Kosmidis, Kenne Pagui, and Sartori (2020).

The derivatives of the log-likelihood, known as the **score functions**, about β and ϕ are

$$s_{\beta} = \frac{1}{\phi} X^T W D^{-1} (y - \mu), \quad s_{\phi} = \frac{1}{2\phi^2} \sum_{i=1}^n (q_i - \rho_i)$$

with $\mathbf{y} = (y_1, \dots, y_n)^T$, $\mu = (\mu_1, \dots, \mu_n)^T$, $W = \text{diag}\{w_1, \dots, w_n\}$, and $D = \text{diag}\{d_1, \dots, d_n\}$, where $w_i = m_i d_i^2 / v_i$ is the i th working weight, with $d_i = d\mu_i / d\eta_i$ and $v_i = V(\mu_i)$. Furthermore, $q_i = -2m_i \{y_i \theta_i - b(\theta_i) - c_1(y_i)\}$ and $\rho_i = m_i a'_i$ with $a'_i = a'(-m_i / \phi)$.

The expected **information matrix** about β and ϕ is

$$i = \begin{bmatrix} i_{\beta\beta} & \mathbf{0}_p \\ \mathbf{0}_p^T & i_{\phi\phi} \end{bmatrix} = \begin{bmatrix} \frac{1}{\phi} X^T W X & \mathbf{0}_p \\ \mathbf{0}_p^T & \frac{1}{2\phi^4} \sum_{i=1}^n m_i^2 a''_i \end{bmatrix}$$

where $\mathbf{0}_p$ is a p -vector of zeros and $a''_i = a''(-m_i / \phi)$. (Kosmidis, Kenne Pagui, and Sartori 2020)

The core idea behind all of `brglm2`'s estimation approaches is to modify the score equations during fitting rather than computing the MLE and applying a correction afterwards. The standard score vector has a j -th component that is the derivative of the log-likelihood with respect to β_j , and the MLE satisfies $\mathbf{s}_{\hat{\beta}} = \mathbf{0}$. The adjusted score methods instead solve:

$$\mathbf{s}_{\beta}(\beta) + \mathbf{A}_{\beta}^*(\beta) = \mathbf{0}$$

where $\mathbf{A}_{\beta}^*$ is a method-specific correction chosen to eliminate or reduce $b(\beta)$. Each method defines its j -th component as a weighted sum over observations:

$$A_j^* = \sum_{i=1}^n \xi_i x_{ij}$$

where the per-observation weight ξ_i differs by method and depends critically on the hat values h_i , measuring how strongly observation i influences its own fitted value.

The hat values h_i are the diagonal elements of the projection matrix $\mathbf{H} = X_w(X_w^T X_w)^{-1} X_w^T$, where $X_w = W^{1/2} X$ is the weight-scaled design matrix. Via the QR decomposition $X_w = \mathbf{Q}\mathbf{R}$, they are $h_i = \sum_j Q_{ij}^2$, computed in $\mathcal{O}(np^2)$ per iteration. Every adjusted score method requires h_i at each step; any reduction in this cost benefits all methods simultaneously.

2.3.1 Mean Bias Reduction: `AS_mean`

Firth (1993) proposed eliminating the leading $\mathcal{O}(n^{-1})$ bias by choosing A_{β}^* to satisfy $b(\beta) = i_{\beta\beta}^{-1} A_{\beta}^*$, which exactly cancels the first-order bias term. Differentiating $i_{\beta\beta} = X^T W X$ through the working weights with respect to β_j and evaluating the resulting trace leads to the j -th component (Firth 1993; Kosmidis, Kenne Pagui, and Sartori 2020):

$$A_j^{*,\text{mean}} = \frac{1}{2} \text{tr} \left(i_{\beta\beta}^{-1} \frac{\partial i_{\beta\beta}}{\partial \beta_j} \right) = \sum_{i=1}^n \frac{h_i}{2} \cdot \frac{d_i''}{d_i' w_i a(\phi)} \cdot x_{ij}$$

where $d_i' = d\mu_i/d\eta_i$, $d_i'' = d^2\mu_i/d\eta_i^2$, and w_i is the working weight.

The trace form on the left makes explicit that the adjustment depends on how the Fisher information changes with β_j ; the per-observation form on the right shows that this reduces to weighting each covariate x_{ij} by the leverage h_i and the local link nonlinearity d_i''/d_i' . For logistic regression with the canonical logit link, $d_i''/d_i' = 1 - 2\mu_i$, recovering Firth's original expression.

2.3.2 Median Bias Reduction: `AS_median`

Kenne Pagui, Salvan, and Sartori (2017) extended Firth's approach to target median rather than mean unbiasedness. Median unbiasedness requires cancelling an additional skewness term in the expansion of the distribution of $\hat{\beta}$; this introduces a second sum involving the component-wise leverage $\tilde{h}_{ij}$:

$$A_j^{*,\text{med}} = \underbrace{\sum_{i=1}^n \frac{h_i}{2} \cdot \frac{d_i''}{d_i' w_i a(\phi)} \cdot x_{ij}}_{= A_j^{*,\text{mean}}} + \sum_{i=1}^n \tilde{h}_{ij} \left(\frac{V'(\mu_i) d_i'}{6 V(\mu_i) a(\phi)} - \frac{d_i''}{2 d_i' w_i a(\phi)} \right) x_{ij}$$

where $V(\mu_i) = b''(\theta_i)$ is the variance function, $V'(\mu_i) = dV/d\mu_i$, and the component-wise leverage is $\tilde{h}_{ij} = w_i \mathbf{x}_i^T (i_{\beta\beta}^{-1})_{[j, \cdot]} \mathbf{x}_i$, the j -th diagonal element of $W^{1/2} X i_{\beta\beta}^{-1} X^T W^{1/2}$ restricted to the j -th row of $i_{\beta\beta}^{-1}$.

Because it requires the full inverse information matrix in addition to the hat values, this is the most computationally intensive method.

2.3.3 Mixed Bias Reduction: `AS_mixed`

The mixed adjustment (Kosmidis, Kenne Pagui, and Sartori 2020), the package default, applies mean bias reduction to the regression parameters β and median bias reduction to the dispersion parameter ϕ when it is estimated. For binomial and Poisson families where ϕ is fixed at 1, `AS_mixed` and `AS_mean` produce identical results and identical A_j^* weights. The distinction arises only for families with a free dispersion parameter (Gaussian, Gamma, inverse Gaussian), where the mixed approach yields mean-unbiased regression coefficients alongside a median-unbiased dispersion estimate.

2.3.4 Maximum Penalised Likelihood with Jeffreys Prior: `MPL_Jeffreys`

This method (Kosmidis and Firth 2021) maximises the log-likelihood penalised by $p(\beta) = a \log |i_{\beta\beta}|$. Differentiating with respect to β_j gives $\partial p / \partial \beta_j = a \operatorname{tr}(i_{\beta\beta}^{-1} \partial i_{\beta\beta} / \partial \beta_j)$, which is exactly $2a$ times the trace expression defining $A_j^{*,\text{mean}}$ above. The adjusted score from the penalised objective is therefore:

$$A_j^{*,J} = 2a \sum_{i=1}^n \frac{h_i}{2} \cdot \frac{d_i''}{d_i' w_i a(\phi)} \cdot x_{ij} = \sum_{i=1}^n a h_i \left(\frac{d_i''}{d_i' w_i a(\phi)} - \frac{V'(\mu_i) d_i'}{V(\mu_i) a(\phi)} \right) x_{ij}$$

where the second term arises from differentiating the variance function through $i_{\beta\beta}$ for non-canonical links.

Setting $a = \frac{1}{2}$ gives $2a = 1$, so `MPL_Jeffreys` with $a = 0.5$ recovers $A_j^{*,\text{mean}}$ exactly. For the canonical logit link the variance term vanishes ($V' = 0$ for binomial), and this reduces to Firth's original formulation. The scalar a controls the degree of shrinkage, and the finiteness properties of the resulting estimators are studied in detail by Kosmidis and Firth (2021).

2.3.5 Asymptotic Bias Correction: `correction`

Rather than an iterative adjusted score approach, `type = "correction"` implements the post-hoc asymptotic bias correction of Cordeiro and McCullagh (1991): $\hat{\beta}_c = \hat{\beta} - b(\hat{\beta})$. This requires only one additional step beyond the MLE and does not involve iterative hat value computation. Because the correction relies on a single application of the Fisher scoring update structure, it is handled separately within the implementation and is not a target for trust region optimisation.

2.3.6 Computational Similarities

All four iterative adjusted score methods depend on hat values h_i at every iteration. Any strategy that reduces the cost of computing h_i benefits all four methods simultaneously, and this shared dependency is the central insight motivating the algorithmic contribution of this project.

2.4 The `brglm2` Computational Bottleneck

`brglm2` (Kosmidis 2023) implements all five estimation approaches within R's standard `glm()` interface. The core fitting routine `brglmFit()` proceeds by quasi-Fisher scoring: at each iteration it (i) computes weighted working residuals, (ii) forms the hat matrix via QR decomposition of X_w , (iii) evaluates the bias adjustment, and (iv) solves the augmented weighted least squares problem. Steps (ii) and (iii) together account for $\mathcal{O}(np^2)$ work per iteration.

To establish precisely where time is spent, detailed profiling was conducted using `profvis` (Chang and Luraschi 2023) and `microbenchmark` (Mersmann 2023). Variants of the algorithm with progressively fewer linear-algebraic components were benchmarked, yielding the following execution times on a representative problem (MultipleFeatures, $n = 2000$, $p = 140$):

Table 1: Profiling breakdown (50 evaluations, MultipleFeatures, $n = 2000$, $p = 140$).

Component	Min (ms)	Median (ms)	Max (ms)
QR decomposition + hat values	40.59	50.94	140.36
QR decomposition only	16.89	21.42	36.79
No QR	1.25	1.74	13.12

The pattern is striking: removing QR entirely reduces median execution time by a factor of almost thirty. Over 85% of total execution time is consumed by QR-based operations, and hat value calculation accounts for a substantial fraction beyond the QR decomposition alone. The profiling flame graphs are provided in Appendix A. These results directly motivate the algorithmic contributions of this project, any method that avoids or amortises full QR decomposition at every iteration stands to achieve substantial performance gains.

2.5 Trust Region Optimisation

The choice of trust region optimisation over gradient descent alternatives reflects a theoretical compatibility argument specific to bias-reduced GLMs; this section develops that argument and introduces the mathematical machinery used in the implementation.

2.5.1 Motivation: Why Not Quasi-Fisher Scoring or Gradient Descent?

The original project specification proposed gradient-based optimisation methods, including Adam (Kingma and Ba 2015) and L-BFGS (Nocedal and Wright 2006), as alternatives to quasi-Fisher scoring. While such methods can be applied by reformulating the problem as the minimisation of a scalar objective, this treats the problem as one of general optimisation rather than structured root-finding based on the adjusted score equations.

Quasi-Fisher scoring, by contrast, directly targets the system

$$\mathbf{r}(\beta) = \mathbf{s}_\beta + \mathbf{A}_\beta^* \mathbf{0},$$

but lacks any mechanism for controlling step size. As a result, when the Fisher information matrix is ill-conditioned or the current iterate is far from the solution, the method may take large and unstable steps.

Gradient-based methods do not fully address this issue. Although they introduce indirect step-size control, they do not exploit the structure of the estimating equations and provide no explicit measure of step reliability.

Trust region methods (Nocedal and Wright 2006, chap. 4) resolve these limitations by combining Newton-type updates with an explicit step control mechanism. By restricting each step to a region where a local model is trusted, and adapting this region based on observed performance, they provide improved robustness in challenging settings.

2.5.2 The Trust Region Subproblem

Trust region methods require a scalar objective. The adjusted score equations $\mathbf{r}(\beta) = \mathbf{s}_\beta + \mathbf{A}_\beta^* \mathbf{0}$ are not themselves the gradient of a standard objective, but they can be made one by defining $f(\beta) = \frac{1}{2} \|\mathbf{r}(\beta)\|^2 \geq 0$. Finding a root of $\mathbf{r}$ is equivalent to minimising f to zero, and this formulation provides the required scalar objective without altering the statistical fixed point.

The gradient of $f(\beta) = \frac{1}{2}\|\mathbf{r}\|^2$ with respect to β is $\nabla f = \mathbf{J}^T \mathbf{r}$, where $\mathbf{J} = \partial \mathbf{r} / \partial \beta$ is the $p \times p$ Jacobian of the adjusted score residual $\mathbf{r} = \mathbf{s}_\beta + \mathbf{A}_\beta^*$. Differentiating the score contribution alone gives

$$\frac{\partial \mathbf{s}_\beta}{\partial \beta} = -X^T W X = -i_{\beta\beta}$$

the negative Fisher information matrix. The adjustment $\mathbf{A}_\beta^*$ contributes an additional $\mathcal{O}(n^{-1})$ term to $\mathbf{J}$ (involving third derivatives of the link function), which is negligible relative to $i_{\beta\beta}$ as n grows. Approximating $\mathbf{J} \approx -i_{\beta\beta}$ therefore introduces an error of the same order as the bias correction itself, and is consistent with the overall $\mathcal{O}(n^{-1})$ accuracy of the adjusted score approach.

The exact Hessian of f is $\mathbf{J}^T \mathbf{J} + \sum_k r_k \nabla^2 r_k$; the Gauss-Newton approximation drops the second-order term, which is small near the solution where $\|\mathbf{r}\| \approx 0$. Substituting $\mathbf{J} \approx -i_{\beta\beta}$ then gives:

$$\mathbf{B} \approx \mathbf{J}^T \mathbf{J} \approx (-i_{\beta\beta})^T (-i_{\beta\beta}) = i_{\beta\beta}^T i_{\beta\beta}$$

Since $i_{\beta\beta} = X^T W X$ is symmetric and positive semi-definite, this product is also positive semi-definite by construction, regardless of whether $i_{\beta\beta}$ is itself invertible. This is a critical property: it means the trust region subproblem is always well-posed even under near-separation, where $i_{\beta\beta}$ may be singular and quasi-Fisher scoring breaks down.

Trust region subproblem (Nocedal and Wright 2006, Algorithm 4.1 & Algorithm 11.5): The step p_k is found by deriving the approximate solution to the subproblem

$$\min_p m_k(p) = f_k + \mathbf{g}_k^T p + \frac{1}{2} p^T \mathbf{B}_k p \quad \text{subject to} \quad \|p\| \leq \Delta_k$$

where $f_k = f(\beta^{(k)})$; $\mathbf{g}_k = \mathbf{J}^T \mathbf{r}_k$ is the gradient of the objective; $\mathbf{B}_k = \mathbf{J}^T \mathbf{J}$ is the Gauss-Newton Hessian approximation (always positive semi-definite); and $\Delta_k > 0$ is the trust region radius. As such we equivalently write our subproblem as

$$\min_p m_k(p) = f_k + p^T \mathbf{J}_k^T p + \frac{1}{2} p^T \mathbf{J}_k^T \mathbf{J}_k p \quad \text{subject to} \quad \|p\| \leq \Delta_k$$

The step p_k is accepted and $\beta^{(k+1)} \leftarrow \beta^{(k)} + p_k$ when the reduction ratio

$$\rho_k = \frac{\|r(x_k)\|^2 - \|r(x_k + p_k)\|^2}{\|r(x_k)\|^2 - \|r(x_k) + \mathbf{J}_k p_k\|^2}$$

exceeds a threshold $\eta \in [0, \frac{1}{4})$. Large ρ_k indicates the quadratic model is accurate and Δ_k is expanded; small ρ_k indicates a poor model and Δ_k is contracted, with the step rejected.

2.5.3 Subproblem Solver Selection: CG-Steihaug

Three solvers for the trust region subproblem were studied: the More-Sorensen exact method (Moré and Sorensen 1983), the dogleg approximation (Nocedal and Wright 2006, Procedure 11.6), and CG-Steihaug (Steihaug 1983; Nocedal and Wright 2006, Algorithm 7.2). The choice between them carries significant consequences for computational cost.

Both the More-Sorensen and dogleg methods require forming $\mathbf{B}_k$ explicitly at cost $\mathcal{O}(np^2 + p^3)$, recovering the same asymptotic scaling as the original algorithm. Dogleg additionally requires $\mathbf{B}_k$ to be positive definite, which fails under separation or rank deficiency, precisely the cases where bias reduction is most valuable. The More-Sorensen method is exact but introduces an iterative Lagrange multiplier search with $\mathcal{O}(p^3)$ cost per inner iteration, making it computationally prohibitive for large p .

CG-Steihaug was therefore selected as the subproblem solver for two reasons.

The first is that it requires only matrix-vector products with $\mathbf{B}_k$ rather than an explicit factorisation, avoiding the $\mathcal{O}(p^3)$ solve required by More-Sorensen and the $\mathcal{O}(np^2)$ formation required by both More-Sorensen and dogleg. In this implementation $\mathbf{J}_k = i_{\beta\beta}$ is already formed by `compute_fit`, so the product is computed directly at $\mathcal{O}(p^2)$ per CG iteration. With typical convergence in $k_{\text{CG}} \ll p$ iterations the total subproblem

cost is $\mathcal{O}(k_{\text{CG}} \cdot p^2)$, compared to $\mathcal{O}(p^3)$ for a direct solve. The matrix-free alternative via $X^T(\mathbf{W}(X\mathbf{d}))$ takes $\mathcal{O}(np)$ per iteration.

The second is that it handles a singular or indefinite Hessian gracefully, preserving correctness under separation. When a CG direction of zero or negative curvature is encountered, CG-Steihaug terminates immediately at the trust region boundary rather than failing. This robustness is critical for bias-reduced GLMs, where near-separation causes the Fisher information to become nearly singular and where other solvers either fail outright or require additional regularisation.

3 Design and Implementation

This chapter describes the technical contributions of the project: the development environment and workflow, the refactoring of `brglmFit`'s core logic into a modular `compute_fit` helper, the trust region outer loop and CG-Steihaug solver, adaptive hat value scheduling, sparse matrix dispatch, and backward compatibility.

3.1 Development Environment and Workflow

The first milestone was establishing a robust development environment to support logical modification, testing, and performance evaluation. The `brglm2` GitHub repository was forked and linked upstream to Prof. Kosmidis' original repository following Git best practices, maintaining synchronisation with upstream changes while keeping development work isolated on feature branches.

The environment uses VSCode with R 4.5.2 (2025-10-31 UCRT) and the Radian R console for interactive development. The `devtools` package (Wickham et al. 2023) enables loading the local development version with `devtools::load_all()`, while the `tinytest` framework (van der Loo 2020) allows running the complete test suite with `tinytest::test_all()`. This setup proved essential for iterative development with immediate feedback on whether changes break existing functionality. Importantly, `brglm2`'s test suite was already extensively implemented by Prof. Kosmidis prior to this project, comprising 394 tests covering a wide range of edge cases, problem densities, and model families. This comprehensive suite enables a test-driven development methodology throughout.

Development follows a structured Git branching strategy. The `main` branch remains synchronised with upstream; `develop` acts as an integration branch for tested features; all substantial changes are implemented on isolated feature branches (`feat/feature-name`) with descriptive commit messages documenting design decisions. The trust region implementation resides on the `feat/trust-region` branch. Development tasks are tracked using a centralised ticket system, which records identified issues, implementation choices, and progress. This workflow simplifies debugging, enables safe reversion of changes, and provides a clear record of code evolution.

3.2 Codebase Architecture and Refactoring

The original `brglmFit()` embedded all computational logic in a single monolithic function so two targeted contributions were made. The `compute_fit` helper and Boolean lazy-evaluation flags decompose this into independently testable and benchmarkable components.

3.2.1 The `compute_fit` Helper

The original `brglmFit()` embedded weighted residual computation, QR decomposition, hat value calculation, gradient assembly, and dispersion estimation within a single function `key_quantities`. Code analysis revealed substantial duplication and calculations that could be optimised or eliminated. This original structure also precluded targeted profiling analysis and clean substitution of alternative optimisation algorithms.

A new helper function `compute_fit()`, implemented in `brglmFit-helper.R`, extracts and modularises this logic. The function accepts the current parameter vector (β, ϕ) , the design matrix, response, weights, offset, and family object, and returns all quantities required by the outer iteration loop as a named list of class `brglmFit_quantities`: fitted values $\hat{\mu}_i$, working weights w_i , hat values h_i , the QR decomposition of X_w , pre-computed gradient components, and dispersion-related quantities. The helper module also contains separate implementations of each bias-reduction adjustment (`AS_mean_adjustment`, `AS_median_adjustment`, `AS_mixed_adjustment`, `AS_Jeffreys_adjustment`), dispersion estimation logic for non-canonical families, and utility functions for symbolic differentiation.

The refactored code passes all 394 tests in the existing `tinytest` suite, with performance within $\pm 2\%$ of the original across all standard benchmarks (Appendix B.1-B.3), confirming that correctness is preserved while the codebase becomes substantially more maintainable. This initial refactoring is then merged from its feature branch into the develop branch to serve a clean foundation for implementing alternative optimisation algorithms, as the core iteration logic is now isolated and modular.

3.2.2 Boolean Flags for Lazy Evaluation

`compute_fit` accepts three Boolean control flags, `need_qr`, `need_hatvalues` and `need_inverse`. When `need_qr = FALSE`, the QR decomposition of X_w is skipped entirely; when `need_hatvalues = FALSE`, the hat values are not computed; when `need_inverse = FALSE`, the inverse information is not calculated which is permissible for fixed dispersion families and when the type is not `AS_median`.

3.3 Vectorisation of the Median Bias-Reduction Adjustment

The median bias-reduction adjustment $A_j^{*,\text{med}}$ is the most computationally intensive of the four adjusted score methods because it requires, in addition to the hat values h_i , the full inverse information matrix $i_{\beta\beta}^{-1}$ to form the component-wise leverages $\tilde{h}_{ij}$ (Section 2.3). In the original `brglm2` implementation this is computed inside a loop over $j = 1, \dots, p$. On each iteration the loop constructs a rank-one update of the inverse information:

$$V_j = \frac{\mathbf{v}_j \mathbf{v}_j^T}{v_{jj}}, \quad \mathbf{v}_j = (i_{\beta\beta}^{-1})_{[j,:]}$$

allocates the full $p \times p$ matrix V_j , forms the $n \times p$ product XV_j , and computes a weighted inner product to obtain the scalar b_j . The complete p -vector $\mathbf{b}$ is then assembled entry by entry:

Original looped AS_median adjustment — brglmFit-helper.R

```
b_vector <- numeric(nvars)
for (j in seq.int(nvars)) {
  inverse_info_unscaled_j <- inverse_info_unscaled[j, ]
  vcov_j <- tcrossprod(inverse_info_unscaled_j) / inverse_info_unscaled_j[j]
  hats_j <- .rowSums((x %*% vcov_j) * x, nobs, nvars, TRUE) * fit$working_weights
  b_vector[j] <- inverse_info_unscaled_j %*%
    .colSums(x * (hats_j * c_vec), nobs, nvars, TRUE)
}
```

This loop performs p separate $\mathcal{O}(np^2)$ matrix products and p separate $p \times p$ allocations, with all intermediate objects discarded at the end of each iteration. The total arithmetic is $\mathcal{O}(np^3)$ and the allocation pattern is unfriendly to memory: p temporary $p \times p$ matrices are created and freed in sequence, with no reuse between iterations.

The key observation is that the per-column $p \times p$ matrix V_j and the corresponding $n \times p$ product XV_j are not actually needed individually. What is required is b_j , the j -th element of:

$$\mathbf{b} = \text{diag}\left((i^{-1})^T \left[X^T \text{diag}(w_i c_i) (X i^{-1})^{\circ 3} \right]\right)^{-1}$$

where $(\cdot)^{\circ 3}$ (Hadamard notation) denotes element-wise cubing and $c_i = d'_i d_i^{V'} / (6v_i) - d''_i / (2d'_i)$ is the per-observation skewness weight. Writing $XV = X i_{\beta\beta}^{-1}$ (an $n \times p$ matrix computed once) this simplifies to:

$$b_j = \frac{\sum_{i=1}^n w_i c_i (XV)_{ij}^3}{(i^{-1})_{jj}}$$

which is a column-wise weighted sum of cubes of XV , divided by the diagonal of i^{-1} . The entire $\mathbf{b}$ vector is then:

$$\mathbf{b} = \frac{\text{colSums}(XV^{\circ 3} \odot (w_i c_i))}{\text{diag}(i^{-1})}$$

where $\odot$ denotes broadcasting the length- n weight vector across columns. The vectorised implementation computes this directly:

Vectorised AS_median_adjustment_new — brglmFit-helper.R

```
XV      <- as.matrix(x %*% inverse_info_unscaled) # n x p, computed once
d_V     <- diag(inverse_info_unscaled)           # length-p diagonal
c_vec   <- fit$d1mus * fit$d1varmus / (6 * fit$varmus) -
          0.5 * fit$d2mus / fit$d1mus             # length n
wc      <- fit$working_weights * c_vec
b_vector <- colSums(XV^3 * wc) / d_V              # O(np)
```

The arithmetic cost is $\mathcal{O}(np^2)$ for the single matrix multiply and $\mathcal{O}(np)$ for the subsequent element-wise operations, a reduction from $\mathcal{O}(np^3)$ in the loop. The allocation pattern is also improved: a single $n \times p$ matrix `XV` is formed and reused, and its cube `XV^3` is computed in place. The $\mathcal{O}(np^2)$ floor cannot be avoided: XV has np entries and all are needed, so some $\mathcal{O}(np^2)$ work is inherent to the computation regardless of implementation.

The practical consequence is that the saving scales as $\mathcal{O}(p)$ with each additional column we save one $p \times p$ allocation and one $\mathcal{O}(np^2)$ product, so the vectorised version pulls ahead more sharply as p grows. Benchmarking reported in Section 4.3.3 evidences this clearly and further shows the importance of this vectorised method in reducing overall runtime of fitting median adjustment types.

3.4 Trust Region Framework

The trust region outer loop follows Algorithm 11.5 of Nocedal and Wright (2006) which specialises the generic trust region framework of Algorithm 4.1 from the same material, adapted to the adjusted score objective; the full algorithm is described below with reference to the notation established in Section 2.5.

3.4.1 Objective and Gradient

At each outer iteration k , the algorithm minimises $f(\beta) = \frac{1}{2}\|\mathbf{r}(\beta)\|^2$ where the adjusted score residual is $\mathbf{r} = \mathbf{s}_\beta + \mathbf{A}_\beta^*$. The gradient is $\mathbf{g} = \mathbf{J}^T \mathbf{r}$, computed analytically via the Gauss-Newton Jacobian. At convergence the adjusted score norm is zero, the same condition solved by quasi-Fisher scoring, ensuring statistically identical estimates to the original implementation (Kosmidis, Kenne Pagui, and Sartori 2020; Firth 1993).

3.4.2 Outer Loop and Radius Adaptation

At each outer iteration a candidate step p is computed by the CG-Steihaug solver (Section 3.4.3) and a candidate point $\beta_{\text{cand}}^{(k)} = \beta^{(k)} + p_k$ is evaluated via `compute_fit`. The reduction ratio ρ_k , defined in Section 2.5, then governs two separate and independent decisions: whether to accept the step, and how to update the radius Δ_k .

Accept/reject. The step is accepted, $\beta^{(k+1)} \leftarrow \beta_{\text{cand}}^{(k)}$, if and only if $\rho_k > \eta$, with default $\eta = 0.1$. A step is rejected when $\rho_k \leq \eta$: the iterate is not updated, $\beta^{(k+1)} \leftarrow \beta^{(k)}$, and the next iteration restarts from the same point with a smaller radius. The threshold $\eta = 0.1$ is deliberately loose: it requires only that the model predicted at least 10% of the actual reduction, so that even a moderately poor step is accepted provided it makes genuine progress. In practice, a rejected step most commonly arises not from a numerical failure in `compute_fit` but from a poor step direction, one where the quadratic model is an inaccurate approximation to f over the distance $\|p_k\|$. This occurs most often early in the optimisation, when Δ_k is large and the CG-Steihaug solver is permitted to take long steps that venture outside the region where the Gauss-Newton approximation is reliable.

Radius update. The radius adaptation is independent of the accept/reject decision and is governed solely by how accurately the quadratic model predicted the actual reduction.

Radius adaptation rule, (Nocedal and Wright 2006, Algorithm 11.5):

$$\Delta_{k+1} = \begin{cases} \frac{1}{4}\Delta_k & \rho_k < \eta_s \text{ (shrink)} \\ \min(2\Delta_k, \Delta_{\max}) & \rho_k > \eta_e \text{ and } p_k \text{ on boundary (expand)} \\ \Delta_k & \text{otherwise} \end{cases}$$

with defaults $\eta_s = 0.25$ (shrink threshold), $\eta_e = 0.75$ (expand threshold), $\Delta_{\max} = 10^{10}$, $\Delta_0 = 1.0$.

Crucially, the shrink and accept thresholds are set so that $\eta < \eta_s$: a step with $\eta < \rho_k < \eta_s$ is accepted (it makes progress) but the radius is still shrunk (the model is not accurate enough to trust a larger step next time). Only when $\rho_k \leq \eta$ does the algorithm both reject the step and shrink the radius. This separation means the radius adapts to the local accuracy of the quadratic model independently of whether each individual step is usable, allowing the algorithm to recover from a sequence of poor steps by progressively tightening the region within which the Gauss-Newton approximation is trusted.

3.4.3 The CG-Steihaug Subproblem Solver

At each outer iteration the trust region subproblem is solved by CG-Steihaug (Steihaug 1983; Nocedal and Wright 2006, Algorithm 7.2). The algorithm runs conjugate gradient (CG) iterations on the quadratic model $m_k(p)$ (Section 2.5.2) and terminates early either when the step reaches the trust region boundary or when negative curvature in the Hessian approximation is detected. In the latter case the algorithm steps to the boundary along the current search direction rather than failing, which is the key robustness property that makes CG-Steihaug suitable for near-singular problems such as logistic regression under separation.

The central operation at each inner step j is the matrix-vector product $F_k \mathbf{d}_j$, where $F_k = i_{\beta\beta}$ is the Fisher information matrix already computed and stored by `compute_fit` as `fit$info_beta`. This matrix is passed directly to the solver and the product is formed as a standard stored matrix-vector multiply at cost $\mathcal{O}(p^2)$ per CG iteration:

Stored matvec in CG-Steihaug — `brglmFit.R`

```
Fd <- drop(F_info %*% d)
```

The alternative of a matrix-free approach, reconstructing the product via two passes through X as $X^T(W(Xd))$ at cost $\mathcal{O}(np)$, was evaluated and rejected.

3.4.4 Jacobi Preconditioning

Preconditioning accelerates CG convergence by transforming the linear system to improve its eigenvalue distribution. The number of CG iterations required to reduce the residual by a fixed factor is known to scale with the spectral condition number $\kappa(\mathbf{B}_k) = \lambda_{\max}/\lambda_{\min}$, the ratio of the largest to smallest eigenvalue of the Hessian approximation, where $\lambda_{\max}$ and $\lambda_{\min}$ correspond to the directions of greatest and least curvature in the quadratic model respectively. When predictors have very different scales or the Fisher information is nearly singular, one or more eigenvalues of $\mathbf{B}_k$ approach zero and $\kappa(\mathbf{B}_k)$ becomes large, meaning CG must take many small steps to make progress along the poorly scaled directions before converging.

Nocedal and Wright (2006), Algorithm 5.3 details the preconditioned CG method. A non-singular matrix C defines a change of variables $\hat{p} = Cp$, transforming the system so that the effective Hessian becomes $C^{-T}\mathbf{B}_k C^{-1}$. The preconditioner is $M = C^T C$, which must be symmetric and positive definite by construction. The goal is to choose M such that $\kappa(M^{-1}\mathbf{B}_k) \ll \kappa(\mathbf{B}_k)$, ideally with $M \approx \mathbf{B}_k$ so that $M^{-1}\mathbf{B}_k \approx \mathbf{I}$ and convergence is rapid.

The Jacobi preconditioner $M = \text{diag}(i_{\beta\beta})$ is used here. This choice is attractive for three reasons. First, $i_{\beta\beta}$ is already computed and stored by `compute_fit`, so forming M requires only extracting the diagonal at negligible cost. Second, the diagonal entries $(i_{\beta\beta})_{jj} = \sum_i w_i x_{ij}^2$ are the marginal Fisher information for each coefficient and capture the dominant curvature in each coordinate direction; when predictors are on different scales these entries vary substantially, and dividing by them approximately normalises the problem. Third, applying M^{-1} costs only one $\mathcal{O}(p)$ element-wise division per CG step, a negligible overhead relative to the $\mathcal{O}(p^2)$ matrix-vector product. Near-zero diagonal entries are clamped to a small multiple of machine epsilon to ensure M remains positive definite:

Jacobi preconditioner — `brglmFit.R`

```
d_F <- pmax(diag(F_info), .Machine$double.eps * max(diag(F_info)))
Minv <- function(v) v / d_F
```

In practice the Jacobi preconditioner halves or quarters the number of inner CG iterations relative to unpreconditioned CG, directly reducing the $k_{\text{CG}} \cdot \mathcal{O}(p^2)$ inner-loop cost. More sophisticated choices, such as an incomplete Cholesky factorisation $M \approx \mathbf{LL}^T$, would further reduce $\kappa(M^{-1}\mathbf{B}_k)$ at the cost of a more expensive preconditioner application. This is noted as a direction for future work in Section 6.

3.4.5 Adaptive Inner Tolerance: Eisenstat-Walker

The inner loop tolerance is set adaptively using the Eisenstat-Walker rule (Eisenstat and Walker 1996a, 1996b), specifically Choice 2:

Eisenstat-Walker: Choice 2

$$\varepsilon_k = \min \left(\frac{1}{2}, \gamma \left(\frac{\|\mathbf{r}_k\|^2}{\|\mathbf{r}_{k-1}\|^2} \right)^\alpha \right), \quad \gamma = 0.9, \alpha = 2$$

where $\mathbf{r}_k = \mathbf{s}_{\beta^{(k)}}^*$ is the adjusted score at the current outer iterate.

This prevents over-solving the subproblem early in the optimisation: when $\|\mathbf{r}_k\|$ is large the outer iterate is far from the solution, and spending many CG iterations to solve the subproblem to high accuracy is wasted work since the quadratic model itself is a poor approximation to f at that distance. As $\|\mathbf{r}_k\|$ decreases the tolerance tightens automatically, ensuring superlinear convergence near the solution is not compromised.

3.4.6 Convergence Criterion

The algorithm reports convergence when both

$$\max_j |p_{k,j}| < \varepsilon \quad \text{and} \quad |v_\zeta^{(k)}| < \varepsilon$$

where ε is the tolerance from `brglmControl()`, $p_{k,j}$ are the components of the accepted beta step, and $v_\zeta^{(k)}$ is the dispersion Newton step described below. The second condition is omitted for families with fixed dispersion. This criterion matches the step-size convergence condition of the original quasi-Fisher implementation, ensuring equivalent stopping behaviour.

3.5 Dispersion Estimation

Bias-reduced estimation for families with a free dispersion parameter, Gaussian, Gamma, and inverse Gaussian, requires a separate update rule for ϕ ; this section describes the Newton step-halving approach used and the injection that avoids a redundant `compute_fit` call.

3.5.1 Newton Step-Halving for Dispersion

The dispersion parameter ϕ is updated separately from β using a Newton step on the transformed scale $\zeta = T(\phi)$, where T is the transformation specified in `brglmControl()`. A step-halving inner loop ensures positivity: starting from the full Newton step $v_\zeta^{(k)}$, the step is halved up to `max_step_factor` times until $\phi^{(k+1)} = T^{-1}(\zeta^{(k+1)})$ is finite and positive. If positivity cannot be achieved within the allowed halvings, the algorithm reverts to $\zeta^{(k)}$.

An earlier implementation used a hard clamp $\phi \leftarrow \max(\phi, \varepsilon_M)$, where ε_M is machine epsilon. This caused the dispersion to collapse to 2.2×10^{-308} under identity transformations, producing log-scale underflow errors. The step-halving approach resolves this without imposing a floor on ϕ .

3.5.2 Analytical Geometry Injection

After a dispersion update, the Fisher information and its inverse must be updated before the next outer iteration. A full `compute_fit` call at the new $(\beta, \phi^{(k+1)})$ would recompute all quantities including the QR decomposition, at cost $\mathcal{O}(np^2)$. However, since the working weights depend on β and the link but not on

ϕ for canonical families, and since the Fisher information scales as $i_{\beta\beta} \propto \phi^{-1}$, the updated matrices can be obtained by analytical rescaling:

$$i_{\beta\beta}^{(k+1)} = i_{\beta\beta}^{(k)} \cdot \frac{\phi^{(k)}}{\phi^{(k+1)}}, \quad \left(i_{\beta\beta}^{(k+1)}\right)^{-1} = \left(i_{\beta\beta}^{(k)}\right)^{-1} \cdot \frac{\phi^{(k+1)}}{\phi^{(k)}}$$

Hat values and the QR decomposition are carried over unchanged, as they depend only on β through the working weights and are refreshed by the periodic hat value update schedule described in Section 3.6. This geometry injection reduces the per-dispersion-update cost from $\mathcal{O}(np^2)$ to $\mathcal{O}(p^2)$.

3.6 Hat Value Computation

Hat values are both statistically essential and the dominant computational cost; this section describes the exact computation, the adaptive scheduling strategy, and the design decision to prefer this over stochastic approximations.

3.6.1 Exact Updates via QR Decomposition

Hat values $h_i = \sum_j Q_{ij}^2$ are the squared row-norms of the orthogonal factor from the QR decomposition of the weighted design matrix $X_w = W^{1/2}X$, computed in `brglmFit-helper.R` as:

Hat value computation via QR decomposition — `brglmFit-helper.R`

```
wx      <- sqrt(working_weights) * x
qr_decomp <- qr(wx)
Q_matrix <- qr.Q(qr_decomp)
hatvalues <- .rowSums(Q_matrix * Q_matrix, nobs, nvars, TRUE)
```

This exact computation costs $\mathcal{O}(np^2)$ per call and, when invoked at every iteration, constitutes the primary bottleneck identified in profiling. Reducing its frequency is the single most impactful optimisation available.

3.6.2 Adaptive Scheduling Strategy

Since hat values change slowly during convergence, particularly once the parameter vector is close to the solution, recomputing them at every iteration is unnecessary. The adaptive scheduling strategy defers exact hat value recomputation to every F_h -th accepted step, using values from the most recent exact computation in intervening steps. The parameter F_h is exposed as `hat_update_freq` in `brglmControl()`, defaulting to 1 (a recomputation at every accepted step) to preserve the behaviour of the original implementation when no value is supplied.

A key implementation detail is that the periodic refresh exploits the QR decomposition already stored by `compute_fit`: calling `qr.Q()` on the stored `qr_decomposition` object recomputes **Q** without a second QR factorisation. The hat values are then recomputed as squared row-norms of **Q** at cost $\mathcal{O}(np)$ rather than $\mathcal{O}(np^2)$, since the expensive triangularisation step is already done.

Periodic hat value refresh — brglmFit.R

```
if (hat_accept_count %% hat_update_freq == 0L) {
  Q_tmp      <- qr.Q(fit$qr_decomposition)
  hatvalues_cached <- .rowSums(Q_tmp * Q_tmp, nobs, nvars, TRUE)
  Q_tmp      <- NULL
  fit$hatvalues <- hatvalues_cached
}
```

The matrix $\mathbf{Q}$ is an $n \times p$ object and is freed immediately after the hat values are extracted to avoid unnecessary memory retention.

3.6.3 Rejected Alternative: Stochastic Approximations

Stochastic diagonal estimation via Hutchinson’s method (Hutchinson 1990) was evaluated but rejected. The non-deterministic noise it introduces can trigger spurious convergence failures and is incompatible with the package’s requirement for reproducible results at fixed random seeds.

The adaptive scheduling strategy is therefore preferred over both alternatives: it is exact when invoked, deterministic, and can be tuned continuously from the original every-iteration behaviour down to infrequent updates for large-scale problems.

3.7 Sparse Matrix Support

Many large-scale applications of bias-reduced GLMs involve sparse design matrices. Examples include text mining with bag-of-words representations, genomic SNP data with binary allele indicators, and high-dimensional epidemiological studies, all of which can exhibit sparsity exceeding 80%. While the sparsity patterns in these settings are not identical, they can be approximately mimicked using synthetic constructions such as one-hot encoded categorical variables, interaction dummy matrices, and block-diagonal structures arising from grouped data. These synthetic designs reproduce comparable levels of sparsity and some aspects of the underlying structure, providing a useful proxy for real-world sparse problems.

When the design matrix is detected as a `dgCMatrix` (compressed sparse column format from the `Matrix` package (Bates, Maechler, and Jagan 2024)) or when the provided design matrix is suitably sparse, `brglmFit` automatically dispatches to a sparse-aware computational path without any change to the user-facing interface. Sparse detection is performed before the `as.matrix()` coercion that the dense path requires, preserving the sparse representation throughout fitting. The threshold for dispatch is a fill rate below 20% (above 80% sparsity) with more than 20 columns, chosen to ensure sparse arithmetic is consistently faster than dense.

3.7.1 Cholesky Factorisation for Hat Values

The dense path computes hat values via QR decomposition of the $n \times p$ weighted design matrix $X_w = W^{1/2}X$, at cost $\mathcal{O}(np^2)$ and requiring an $n \times p$ dense intermediate matrix to be formed. For sparse X this intermediate is dense regardless of the sparsity of X , forfeiting all sparsity benefits. The sparse path instead forms $X^T W X$ directly at cost $\mathcal{O}(\text{nnz}(X))$ using slot-level row scaling, then computes its Cholesky factorisation $R^T R = X^T W X$ at cost $\mathcal{O}(p^3)$. Hat values are then recovered as:

$$h_i = w_i \|R^{-T} X_i\|^2$$

computed via `backsolve(R_matrix, t(Xd), transpose = TRUE)`, which solves the lower triangular system $R^T Z = X_d^T$ to give a $p \times n$ matrix Z whose column norms yield the hat values as `colSums(Z * Z)`. The `transpose = TRUE` argument is critical: without it the upper triangular solve computes R^{-1} rather

than R^{-T} , producing incorrect hat values.

Row scaling of a sparse matrix requires some care as R's `Matrix` package does not broadcast a length- n vector row-wise across an $n \times p$ `sparseMatrix` in the same way as base R with dense matrices. Also, using `Diagonal(x = w) %*% x` allocates an $n \times n$ sparse object with full S4 overhead. Instead, the sparse weighting is performed at the slot level, operating directly on the `@x` and `@i` slots of the `CsparseMatrix`:

Slot-level sparse row scaling — `brglmFit-helper.R`

```
.sparse_row_scale <- function(x, w) {  
  cx <- x  
  cx@x <- cx@x * w[cx@i + 1L]  
  cx  
}
```

This is $\mathcal{O}(\text{nnz}(X))$ with no object allocation. The same principle applies to the Jacobi preconditioner diagonal, which uses `colSums(S * S)` where `S = .sparse_row_scale(x, sqrt(w))` rather than extracting the diagonal of the explicitly formed $X^T W X$.

3.8 Backward Compatibility

Backward compatibility was treated as a hard constraint throughout development. The function signatures of `brglmFit()` are unchanged, all 394 existing tests pass with the exception of two currently under investigation, and the default behaviour (`type = "AS_mixed"`) produces numerically identical results to the original implementation to within numerical tolerance. Users experience no change in default behaviour with the trust region method integration plan to be opt-in and undergo further testing scrutiny before being pushed to the package build.

Of the two failing tests, one relates to the `correction` type. Because the bias correction of Cordeiro and McCullagh (1991) is a single closed-form step rather than an iterative adjusted score procedure, it is not a natural target for the trust region framework. The fix requires computing the correction directly as a single Newton step using the adjustment vector and information matrix, bypassing the trust region loop entirely, and remains as future work.

3.9 Algorithm Summary

The complete algorithm implemented in `brglmFit.R` is described below in structured pseudocode. Throughout, superscripts in parentheses denote outer iteration indices, e.g. $\beta^{(k)}$ is the value of β at the k -th outer iteration. The adjusted score for β, ϕ evaluated at $(\beta^{(k)}, \phi^{(k)})$ is written

$$s_{\beta^{(k)}}^* := s_{\beta} + A_{\beta} \quad \text{and} \quad s_{\phi^{(k)}}^* := s_{\phi} + A_{\phi}$$

The Fisher information matrices evaluated at $(\beta^{(k)}, \phi^{(k)})$ are $i_{\beta\beta}^{(k)}$ and $i_{\phi\phi}^{(k)}$.

The following are inputs to the algorithm:

- $s_{\beta}^*, i_{\beta\beta}$ - adjusted score and Fisher information for the regression coefficients
- $s_{\phi}^*, i_{\phi\phi}$ - adjusted score and Fisher information for the dispersion parameter
- Starting values $\beta^{(0)}, \phi^{(0)}$
- $\epsilon > 0$ convergence tolerance (on the L^{∞} norm of the step)
- Trust region parameters:
 - Starting region radius $\Delta_0 = 1$ with $\Delta_k \in (0, \Delta_{\max})$
 - Accept condition $\eta \in [0, \frac{1}{4})$
 - Shrink/expand thresholds $\frac{1}{4}$ and $\frac{3}{4}$; shrink factor $\frac{1}{4}$, expand factor 2
- Hat value recomputation frequency F_h (every F_h accepted steps)
- K the maximum number of outer iterations

The algorithm outputs the following

$$\tilde{\beta}, \tilde{\phi}$$

Outer loop initialisation

1. $k \leftarrow 0$
2. Set step tolerance via Eisenstat-Walker adaptive tolerance

$$\epsilon_k = \min \left(\frac{1}{2}, \gamma \left(\frac{\|s_{\beta_k}^*\|^2}{\|s_{\beta_{k-1}}^*\|^2} \right)^{\alpha} \right) \quad \gamma = 0.9, \alpha = 2$$

CG-Steihaug inner loop initialisation

3. $\epsilon_k > 0$ from (2)
4. $z_0 = 0_p$
5. $r_0 = s_{\beta^{(k)}}^*$
6. $y_0 = M^{-1}r_0$ with $M = \text{diag}(i_{\beta\beta}^{(k)})$ (Jacobi Preconditioning)
7. $d_0 = y_0$
8. $j \leftarrow 0$

CG-Steihaug inner loop (Nocedal & Wright, Algorithm 7.2)

9. If $j \geq j_{\max}$, go to step 21 with $p_k = z_j$.
10. Compute $\kappa_j = d_j^T i_{\beta\beta}^{(k)} d_j$.
11. If $\kappa_j \leq 0$ (zero or negative curvature detected):
 - 11.1 Find $\tau \geq 0$ such that $p_k = z_j + \tau d_j$ satisfies $\|p_k\| = \Delta_k$.
 - 11.2 Go to step 21.
12. $\alpha_j \leftarrow (r_j^T y_j) / \kappa_j$.
13. $z_{j+1} \leftarrow z_j + \alpha_j d_j$.
14. If $\|z_{j+1}\| \geq \Delta_k$ (trust region boundary reached):
 - 14.1. Find $\tau \geq 0$ such that $p_k = z_j + \tau d_j$ satisfies $\|p_k\| = \Delta_k$.
 - 14.2. Go to step 21.
15. $r_{j+1} \leftarrow r_j - \alpha_j i_{\beta\beta}^{(k)} d_j$.
16. $y_{j+1} \leftarrow M^{-1} r_{j+1}$.
17. If $\sqrt{r_{j+1}^T y_{j+1}} < \varepsilon_k \cdot \sqrt{r_0^T y_0}$:
 - 17.1. Go to step 21 with $p_k = z_{j+1}$.
18. $\beta_j \leftarrow (r_{j+1}^T y_{j+1}) / (r_j^T y_j)$.
19. $d_{j+1} \leftarrow y_{j+1} + \beta_j d_j$.
20. $j \leftarrow j + 1$, go to step 9.

Outer loop trust region modification (Nocedal & Wright, Algorithm 4.4/11.5)

21. Predicted reduction under step p_k :

$$\text{predicted}_k = \|s_{\beta^{(k)}}^*\|^2 - \|s_{\beta^{(k)}}^* - i_{\beta\beta}^{(k)} p_k\|^2.$$

22. $\beta_{\text{cand}}^{(k+1)} \leftarrow \beta^{(k)} + p_k$.
23. $s_{\beta_{\text{cand}}^{(k+1)}}^* \leftarrow s_{\beta_{\text{cand}}^{(k+1)}} + A_{\beta_{\text{cand}}^{(k+1)}}$.
24. Actual reduction:

$$\text{actual}_k = \|s_{\beta^{(k)}}^*\|^2 - \|s_{\beta_{\text{cand}}^{(k+1)}}^*\|^2.$$

25. Reduction ratio:

$$\rho_k = \frac{\text{actual}_k}{\text{predicted}_k}.$$

- 26.

$$\Delta_{k+1} \leftarrow \begin{cases} \frac{1}{4} \Delta_k & \text{if } \rho_k < \frac{1}{4}, \\ \min(2 \Delta_k, \Delta_{\max}) & \text{if } \rho_k > \frac{3}{4}, \\ \Delta_k & \text{otherwise.} \end{cases}$$

27. If $\rho_k > \eta$ (Accept step)

- 27.1. $\beta_{\text{cand}}^{(k+1)} \leftarrow \beta_{\text{cand}}^{(k)}$
- 27.2. $s_{\beta_{\text{cand}}^{(k+1)}}^* \leftarrow s_{\beta_{\text{cand}}^{(k+1)}}^*$.

- 27.3. Increment accepted-step counter. If the counter is a multiple of F_h , recompute hat values $h_i^{(k+1)}$.

28. If $\rho_k \leq \eta$ (Reject step):

- 28.1. $\beta_{\text{cand}}^{(k+1)} \leftarrow \beta^{(k)}$
- 28.2. $s_{\beta_{\text{cand}}^{(k+1)}}^* \leftarrow s_{\beta^{(k)}}^*$.

Dispersion inner loop

29. Compute the Newton step on the ζ scale:

$$v_{\zeta}^{(k)} \leftarrow \left(i_{\zeta\zeta}^{(k)} \right)^{-1} s_{\zeta^{(k)}}^*.$$

30. Step-halving (initialise $m \leftarrow 0$):

30.1. $\zeta^{(k+1)} \leftarrow \zeta^{(k)} + 2^{-m} v_{\zeta}^{(k)}.$

30.2. $\phi^{(k+1)} \leftarrow T^{-1}(\zeta^{(k+1)}).$

30.3. If $\phi^{(k+1)} > 0$ and is finite, accept; otherwise $m \leftarrow m + 1$ and go to 30.1 (up to M halvings).

31. Update the fit at $(\beta^{(k+1)}, \phi^{(k+1)})$ for use in the next outer iteration. Fisher information matrix entries that scale linearly with ϕ are rescaled analytically to avoid a full recomputation:

$$i_{\beta\beta}^{(k+1)} \leftarrow i_{\beta\beta}^{(k)} \cdot \frac{\phi^{(k)}}{\phi^{(k+1)}}, \quad \left(i_{\beta\beta}^{(k+1)} \right)^{-1} \leftarrow \left(i_{\beta\beta}^{(k)} \right)^{-1} \cdot \frac{\phi^{(k+1)}}{\phi^{(k)}}.$$

Hat values and the QR decomposition are carried over unchanged, as they depend only on β (through the working weights) and are refreshed by the periodic schedule in step 27.3.

Convergence and termination

32. Check convergence. The algorithm reports convergence when both

$$\max_j |p_{k,j}| < \epsilon \quad \text{and} \quad |v_{\zeta}^{(k)}| < \epsilon,$$

where the second condition is omitted for families with fixed dispersion (binomial, Poisson). If convergence is not achieved within K outer iterations, a warning is issued.

33. **Outputs:** $\tilde{\beta} \leftarrow \beta^{(k)}, \tilde{\phi} \leftarrow \phi^{(k)}.$

4 Testing and Validation

This chapter describes the benchmarking infrastructure established to evaluate performance, the correctness criteria used to validate the trust region implementation, and the results of both the standard package benchmarks and the synthetic suite.

4.1 Benchmarking Infrastructure

A comprehensive benchmarking framework was established prior to any substantive algorithmic changes. This ordering was deliberate: without a reliable baseline it is impossible to distinguish genuine performance improvements from system noise. Two complementary benchmark suites were developed, targeting different aspects of the implementation.

The first is `main_benchmark.R`, which evaluates the standard package datasets used by `brglm2`'s own documentation and vignettes: Lizards (small, $n = 24$, $p = 5$, binomial logistic), Endometrial probit (small-medium, $n = 79$, $p = 4$), and MultipleFeatures (high-dimensional, $n = 1000$, $p = 140$, binomial logistic following Sterzinger and Kosmidis 2024). These use `microbenchmark` (Mersmann 2023) with 5000 and

2500 replications respectively for the small datasets, and `bench::mark` with 150 interleaved iterations for MultipleFeatures. The `bench` approach (Hester and Vaughan 2025) is preferred for larger problems because it interleaves evaluations of both implementations rather than running all original iterations followed by all new ones, thereby eliminating bias from thermal throttling and memory state. Warm-up fits are performed before timing begins and garbage collection is forced between sections. This suite runs exclusively under the package default `type = "AS_mixed"` and serves as the primary test of correctness and stability on known real-world datasets. Results from all runs are automatically logged to timestamped files recording full system metadata, enabling longitudinal performance tracking across development iterations.

The second is `trust_test.R`, a synthetic benchmark suite covering thirteen dense problem configurations across four GLM families and link functions, plus seven sparse configurations. All scenarios were executed under both `type = "AS_mixed"` and `type = "AS_median"` to isolate the contribution that the adjustment type makes to computational performance relative to the original. For the dense scenarios, `AS_mixed` was run with 30 replications per scenario while both `AS_median` variants were run with 15. For the sparse scenarios, `AS_mixed` was run with 15 replications and `AS_median` with 5; the reduction reflects the prohibitive runtime of the original implementation at high sparsity, particularly under median bias reduction, where a single fit can take several minutes or hours. All results are produced within a single R session on identical datasets, ensuring that observed differences are attributable to implementation rather than system variation.

4.2 Correctness Validation Against `brglmFit_original()`

Correctness is validated by comparing the trust region implementation against the original quasi-Fisher scoring implementation, accessed via `method = "brglmFit_original"` within the fitting interface. This comparison is the appropriate reference standard for two complementary reasons. Practically, `brglm2` has been publicly available on CRAN since 2017 (Kosmidis 2023) and accumulates approximately one million downloads annually; after nearly nine years of use across statistics, medicine, epidemiology, and the social sciences the community of users has stress-tested the implementation across an exceptionally wide range of model specifications and edge cases, with issues reported to and addressed by the package author. The original implementation can therefore be treated as a numerically validated reference. Technically, the comprehensive `tinytest` suite built up by Prof. Kosmidis operationalises this validation: any changes to the system must ensure that all existing tests continue to pass, providing a concrete and quantitative standard against which the new implementation is held.

Coefficient agreement is assessed via the maximum absolute difference, $\max_j |\hat{\beta}_j^{\text{new}} - \hat{\beta}_j^{\text{original}}|$, across all scenarios and replications. For the synthetic suite under `AS_mixed`, all thirteen dense scenarios achieve differences below 9×10^{-7} , with most below 5×10^{-7} ; the seven sparse configurations fall uniformly below 6×10^{-7} . The `AS_median` results are consistent with these figures: all dense differences lie below 10^{-6} and all sparse scenarios produce differences below 7×10^{-7} . Across both adjustment types, agreement is well within numerical tolerance for double-precision arithmetic. The results confirm that the trust region implementation recovers the same fixed point as quasi-Fisher scoring, as the theoretical analysis in Section 2.5 predicts. It is important to note that the new vectorised median fitting routine yields coefficient estimates that coincide with those of the original implementation, confirming that the refactoring does not alter the underlying estimator.

A selection of the coefficient agreement results from `trust_test.R` is presented in Table 2 for completeness. The entire table can be found in the Appendix (Table B.5).

Table 2: Coefficient agreement between the trust region implementation and the original quasi-Fisher scoring across all dense synthetic scenarios.

Scenario	n	p	$\max \Delta \hat{\beta} $ (AS_mixed)	$\max \Delta \hat{\beta} $ (AS_median)
Logistic AR-1	200	5	2.42×10^{-8}	5.91×10^{-7}
Logistic AR-1	1000	20	1.87×10^{-7}	2.08×10^{-7}
Logistic AR-1	4000	50	6.14×10^{-9}	6.05×10^{-9}
Logistic AR-1	8000	100	8.22×10^{-8}	8.09×10^{-8}
Probit block-corr	500	10	6.55×10^{-7}	6.89×10^{-7}
Probit block-corr	2000	30	1.85×10^{-7}	1.89×10^{-7}
Probit block-corr	5000	60	3.64×10^{-8}	3.72×10^{-8}
Poisson sparse	500	10	6.86×10^{-7}	5.89×10^{-7}
Poisson sparse	2000	25	8.63×10^{-8}	8.20×10^{-8}
Logistic high-dim	500	100	8.93×10^{-7}	9.67×10^{-7}
Logistic high-dim	1000	200	3.08×10^{-7}	3.13×10^{-7}
Cloglog survival	800	15	1.09×10^{-7}	2.65×10^{-7}
Cloglog survival	3000	40	4.30×10^{-7}	4.48×10^{-7}

Of the 394 tests in the existing `tinytest` suite, 392 pass. The two failures are both understood: the first relates to the `correction` dispatch logic and is straightforward to resolve; the second does not affect any of the bias-reduced adjustment types that are the primary contribution of this project.

4.3 Performance Benchmarks

The refactoring baseline, established at the end of Term 1, confirms that the `compute_fit` abstraction introduces no overhead: performance is within $\pm 2\%$ of the original across all standard benchmarks (Appendix B, Tables B.1-B.3). This baseline serves as the reference point against which all subsequent algorithmic changes are evaluated.

4.3.1 Standard Package Datasets: AS_mixed

Results from `main_benchmark.R` on the standard package datasets indicate that for Lizards and Endometrial, the trust region implementation is modestly slower than the original quasi-Fisher scoring, with speedup factors in the range $0.68\times$ – $1.03\times$ across multiple runs. This is the expected consequence of trust region bookkeeping dominating when the core computation is itself inexpensive. For MultipleFeatures ($n = 1000$, $p = 140$) the results are more variable across runs, with speedups ranging from $0.94\times$ to $1.38\times$. Memory usage shows a more consistent improvement: the MultipleFeatures benchmark reports a reduction from 2.85 GB to 1.95 GB allocated across the fit, a $1.46\times$ reduction attributable to avoiding the repeated $n \times p$ dense intermediate matrices in the QR path.

4.3.2 Synthetic Dense Scenarios: AS_mixed

Timing results from the `AS_mixed` run of `trust_test.R` (30 replications per dense scenario) are presented in Table 3.

Table 3: Dense synthetic timing results under `AS_mixed` (30 replications).

Scenario	n	p	Original median (ms)	New median (ms)	Speedup
Logistic AR-1	200	5	15.15	16.74	0.90×
Logistic AR-1	1000	20	47.71	67.98	0.70×
Logistic AR-1	4000	50	406.30	478.75	0.85×
Logistic AR-1	8000	100	2441.97	2792.58	0.87×
Probit	500	10	26.84	34.41	0.78×
block-corr					
Probit	2000	30	131.31	162.11	0.81×
block-corr					
Probit	5000	60	846.60	834.87	1.01×
block-corr					
Poisson sparse	500	10	80.09	80.95	0.99×
Poisson sparse	2000	25	297.44	306.28	0.97×
Logistic	500	100	241.73	345.15	0.70×
high-dim					
Logistic	1000	200	1869.83	1971.08	0.95×
high-dim					
Cloglog	800	15	63.35	58.32	1.09×
survival					
Cloglog	3000	40	338.27	385.00	0.88×
survival					

Under `AS_mixed` the trust region is typically 10–30% slower than quasi-Fisher scoring on dense problems, with parity approaching at larger n and p . Two observations are worth highlighting. First, at $n = 200$, $p = 5$ the maximum observed time dropped from 1387 ms under the original to 31 ms under the new implementation; this single-run outlier in the original reflects a poor starting condition or ill-conditioned step, and its absence demonstrates the trust region’s resistance to catastrophically large steps. Second, the cloglog scenario at $n = 800$ represents a genuine median speedup of 1.09×, consistent with the trust region engaging more efficiently on link functions with more pronounced nonlinearity.

4.3.3 Synthetic Dense Scenarios: `AS_median`

The `AS_median` runs with 15 replications each and reveal a sharp contrast between the looped and vectorised implementations of the median adjustment. Timing under the looped original adjustment (Table 4) shows that the trust region is substantially *slower* than quasi-Fisher scoring across all but the smallest scenarios: the median overhead at $n = 1000$, $p = 20$ is 2.8× and grows to 2.2× at $n = 4000$, $p = 50$. At $n = 8000$, $p = 100$ the original takes a median of 70.5s while the new implementation takes 161.6s. This is the expected consequence of combining an $\mathcal{O}(np^3)$ adjustment computation with the additional trust region overhead, confirming the analysis in Section 3.3: the trust region alone is insufficient. Vectorisation of the adjustment is necessary for any timing advantage.

Table 4: Dense synthetic timing results under `AS_median` with the looped adjustment (15 replications).

Scenario	n	p	Original median (ms)	New median (ms)	Speedup
Logistic AR-1	200	5	15.04	18.49	$0.81\times$
Logistic AR-1	1000	20	120.53	339.43	$0.36\times$
Logistic AR-1	4000	50	4448.38	9679.35	$0.46\times$
Logistic AR-1	8000	100	70537.82	161599.03	$0.44\times$
Probit	500	10	59.86	92.47	$0.65\times$
block-corr					
Probit	2000	30	723.08	1465.07	$0.49\times$
block-corr					
Probit	5000	60	10284.81	22333.44	$0.46\times$
block-corr					
Logistic high-dim	500	100	6552.11	17950.71	$0.37\times$
Logistic high-dim	1000	200	121004.66	246850.62	$0.49\times$
Cloglog survival	800	15	141.48	187.04	$0.76\times$
Cloglog survival	3000	40	2669.54	5309.57	$0.50\times$

When the vectorised adjustment is combined with the trust region, the picture reverses entirely (Table 5). At the smallest scenarios timings are broadly similar to the original, reflecting the fixed overhead of both the trust region loop and the vectorised matrix multiply at small n and p . Speedups emerge quickly with dimension. At $n = 4000$, $p = 50$ the new implementation reduces median time from 4678 ms to 1114 ms ($4.2\times$). At $n = 8000$, $p = 100$ the original takes a median of 68.8 s; the new implementation completes in 5.8 s, a $11.9\times$ improvement. The high-dimensional logistic scenarios are the most striking: at $n = 500$, $p = 100$ the speedup is $10.0\times$, and at $n = 1000$, $p = 200$ median time falls from 153.1 s to 4.8 s, a $31.6\times$ improvement. These gains are consistent with the $\mathcal{O}(p)$ scaling argument made in Section 3.3. The vectorised adjustment saves a factor proportional to p per iteration, and those savings compound as both n and p increase.

Table 5: Dense synthetic timing results under `AS_median` with the vectorised adjustment (15 replications).

Scenario	n	p	Original median (ms)	New median (ms)	Speedup
Logistic AR-1	200	5	23.45	25.81	$0.91\times$
Logistic AR-1	1000	20	132.29	124.11	$1.07\times$
Logistic AR-1	4000	50	4677.66	1114.47	$4.20\times$
Logistic AR-1	8000	100	68820.81	5762.23	$11.94\times$
Probit	500	10	51.22	49.05	$1.04\times$
block-corr					
Probit	2000	30	712.03	334.90	$2.13\times$
block-corr					
Probit	5000	60	11329.32	1788.93	$6.33\times$
block-corr					
Poisson sparse	500	10	115.10	124.96	$0.92\times$
Poisson sparse	2000	25	727.06	499.01	$1.46\times$

Scenario	n	p	Original median (ms)	New median (ms)	Speedup
Logistic high-dim	500	100	6600.48	658.81	10.02×
Logistic high-dim	1000	200	153111.33	4836.68	31.65×
Cloglog survival	800	15	192.11	118.94	1.61×
Cloglog survival	3000	40	3191.38	853.54	3.74×

4.3.4 Synthetic Sparse Scenarios: AS_mixed

Sparse timing results from `trust_test.R` under `AS_mixed` (15 replications) are presented in Table 6. All seven structural configurations show consistent speedups, ranging from 1.24× to 2.05×

Table 6: Sparse synthetic timing results under `AS_mixed` (15 replications).

Scenario	n	p	Fill rate	Original median (s)	New median (s)	Speedup
One-hot 10 factors × 20 levels	2000	191	0.055	2.07	1.32	1.57×
One-hot 20 factors × 50 levels	5000	981	0.021	251.05	125.00	2.01×
One-hot 5 factors × 100 levels	10000	496	0.012	54.31	35.97	1.51×
Interaction dummies $n_{\text{base}} = 15$	2000	120	0.039	2.00	1.17	1.71×
Interaction dummies $n_{\text{base}} = 20$	5000	210	0.035	20.14	9.84	2.05×
Block diagonal 10 cats × $p = 8$	5000	80	0.100	1.06	0.74	1.43×
Block diagonal 20 cats × $p = 10$	20000	200	0.050	22.06	17.83	1.24×

The largest absolute gains arise on the most extreme sparsity and dimension configurations. The one-hot scenario with 20 factors and 50 levels ($n = 5000$, $p = 981$, fill rate 2.1%) has a median original runtime of 251 seconds; the new implementation completes in 125 seconds. The interaction dummy and block diagonal structures show systematically smaller speedups, reflecting their higher fill rates and lower p , both of which reduce the benefit of the Cholesky-based hat value path relative to the dense QR. The pattern is consistent with the sparsity-aware dispatch described in Section 3.7 with the gains being largest where fill rates are lowest and p is highest.

4.3.5 Synthetic Sparse Scenarios: AS_median

Under AS_median the sparse speedups are dramatically larger. The combination of the vectorised adjustment and the Cholesky-based sparse hat value path produces improvements measured in orders of magnitude for the most extreme configurations (Tables 7 and 8).

The vectorised adjustment results (5 replications) are shown in Table 7. At $n = 2000$, $p = 191$ (one-hot 10 factors) the original takes a median of 122.2 seconds; the new implementation completes in 2.74 seconds, a $44.5\times$ improvement. The one-hot scenario with 5 factors and 100 levels ($n = 10000$, $p = 496$) takes 7202 seconds under the original and 42.7 seconds under the new implementation, a speedup of $168.6\times$. The interaction dummy structures achieve speedups of $25.0\times$ and $47.6\times$ for the two sizes benchmarked, while the block diagonal configurations yield $9.4\times$ and $38.4\times$ respectively.

Table 7: Sparse synthetic timing results under AS_median with the vectorised adjustment (5 replications).

Scenario	n	p	Original median (s)	New median (s)	Speedup
One-hot 10 factors $\times$ 20 levels	2000	191	122.16	2.74	$44.6\times$
One-hot 5 factors $\times$ 100 levels	10000	496	7201.67	42.73	$168.6\times$
Interaction dummies $n_{\text{base}} = 15$	2000	120	43.80	1.75	$25.0\times$
Interaction dummies $n_{\text{base}} = 20$	5000	210	327.27	6.87	$47.6\times$
Block diagonal 10 cats $\times$ $p = 8$	5000	80	6.67	0.71	$9.4\times$
Block diagonal 20 cats $\times$ $p = 10$	20000	200	463.94	12.06	$38.5\times$

Results under the looped adjustment (Table 8), though based on 5 replications with coefficient matching confirmed in each run, tell the same qualitative story. The speedups remain impressive and consistent across structural configurations, ranging from $11.4\times$ to $167\times$. The looped and vectorised results are closely matched here because in the sparse path the dominant cost shifts from the adjustment computation to the Cholesky-based hat value computation; the vectorised adjustment therefore contributes less marginally in the sparse regime than in the dense one.

Table 8: Sparse synthetic timing results under `AS_median` with the looped adjustment (5 replications, coefficient matching confirmed).

Scenario	n	p	Original median (s)	New median (s)	Speedup
One-hot 10 factors $\times$ 20 levels	2000	191	97.26	2.50	38.9 $\times$
One-hot 5 factors $\times$ 100 levels	10000	496	7160.38	42.76	167.5 $\times$
Interaction dummies $n_{\text{base}} = 15$	2000	120	50.69	1.95	26.0 $\times$
Interaction dummies $n_{\text{base}} = 20$	5000	210	431.07	6.33	68.1 $\times$
Block diagonal 10 cats $\times$ $p = 8$	5000	80	15.09	1.32	11.4 $\times$
Block diagonal 20 cats $\times$ $p = 10$	20000	200	537.12	10.74	50.0 $\times$

It is important to note that the one-hot scenario with 20 factors and 50 levels ($n = 5000$, $p = 981$, fill rate 2.1%) was not benchmarked for `AS_median` due to the prohibitive runtime of the original implementation at this scale.

5 Evaluation

This chapter evaluates the project outcomes against the objectives stated at the outset, assesses MoSCoW requirements, summarises computational performance, and identifies limitations.

5.1 Against Project Objectives

The seven project objectives from Section 1.2 are evaluated in turn.

Objective 1 (profiling and exploring bottlenecks) is complete. Profiling established that over 85% of execution time is consumed by QR-based hat value computation, a clear and quantified bottleneck that directly shaped every subsequent design decision.

Objective 2 (refactoring core components) is complete. The `compute_fit` abstraction passes all 394 tests with performance within $\pm 2\%$ of the original, confirming that correctness is preserved and no overhead is introduced by the modularisation.

Objective 3 (trust region solver with CG-Steihaug) is implemented and integrated with all four adjusted score methods, with 392 out of 394 tests passing. The two failures are understood.

Objective 4 (efficient hat value computation) is complete. The adaptive scheduling strategy is implemented and configurable via `hat_update_freq` in `brglmControl()`.

Objective 5 (sparse matrix support) is complete. Sparse matrix dispatch operates automatically, with median speedups ranging from $1.24\times$ to $2.05\times$ under `AS_mixed` and from $9.4\times$ to $168.6\times$ under `AS_median` across all tested sparse structures.

Objective 6 (rigorous benchmarking) is substantially complete. Baseline results confirm the refactoring introduces no overhead. Comparative results are reported across three benchmark configurations. The most significant result is the $31.65\times$ median speedup for `AS_median` at $n = 1000$, $p = 200$, and the $168.6\times$ speedup on the largest sparse scenario benchmarked under the same type.

Objective 7 (production-ready contribution under GPL-3) is on track. The implementation is available at <https://github.com/OllieSClark/brglm2> on branch `feat/trust-region`. Attached to this submission is a build and a README describing installation and reproduction of benchmark results. Backward compatibility is maintained throughout, following the package’s contribution guidelines in collaboration with Prof. Kosmidis.

5.2 MoSCoW Requirements

The original project specification defined a MoSCoW framework of requirements that has been carried forward with modification to reflect the shift from gradient descent to trust region optimisation. All four must-have objectives are complete. MH1 was changed to a trust region approach which was developed and benchmarked in various problem sizes (MH2). Sparse matrix support (MH3) is implemented with automatic dispatch and benchmarked across seven structural configurations. Documentation and testing (MH4) is mostly complete with the `tinytest` unit tests largely passing and the implementation being documented with examples and mathematical pseudocode (Section 3.9).

Of the should-have objectives, extension to Poisson GLMs is evidenced by unit tests and confirmed correct to within 9×10^{-7} (Table 2). Binomial families are covered comprehensively. Adaptive algorithm selection (SH7), the automatic choice between quasi-Fisher scoring for small dense problems and the trust region for large or sparse ones, is the most important outstanding should-have item and is identified as the highest-priority direction for future development. Median bias reduction (SH8) is implemented and substantially outperforms the original once the vectorised adjustment is engaged. The L-BFGS comparison (SH6) was not pursued following the theoretical incompatibility finding described in Section 2.5.

Rcpp integration (CH9) was not attempted due to time constraints, it is a possible avenue for future work however.

5.3 Computational Performance

The headline performance results can be summarised as follows. Under `AS_mixed` on dense problems the trust region implementation is typically 10–30% slower than quasi-Fisher scoring, reflecting the overhead of the outer trust region loop on problems where the adjustment is inexpensive. Under `AS_median` with the looped adjustment the situation is worse: the trust region adds overhead without the compensating reduction in adjustment cost, and the combined implementation is consistently two to three times slower than the original across mid-range dense scenarios. These results demonstrate that neither the trust region alone nor the vectorised adjustment alone is sufficient.

The combination of the trust region with the vectorised adjustment is where the contribution lies. On dense problems the speedup scales with problem dimension, from fairly equal at small n and p to $31.65\times$ at $n = 1000$, $p = 200$. On sparse problems the gains are more dramatic still, as the Cholesky-based hat value computation compounds with the vectorised adjustment to produce improvements of two orders of

magnitude for the largest sparse configurations. At $n = 10000$, $p = 496$ the original implementation takes approximately two hours whilst the new one completes in under a minute.

The improvement in tail behaviour is also noteworthy. Under `AS_mixed` at $n = 200$, $p = 5$, the maximum observed time falls from 1387 ms to 31 ms, a nearly fifty-fold reduction in worst-case runtime. This reflects the trust region’s resistance to catastrophically large steps under ill-conditioning, a property quasi-Fisher scoring does not provide.

5.4 Limitations

Several limitations of the current work should be acknowledged.

The trust region method is yet to be implemented formally within the `brglm2` package. This is due to a number of reasons but primarily hinges on the fact that the strategy of picking which method to use for a given problem is yet to be decided.

Another limitation stems from the fact that not all tests in the test suite are currently passing which inherently means there is some functionality of the original solution that is not developed.

The adaptive hat value scheduling strategy is implemented and configurable, but the optimal frequency parameter is problem-dependent. In a number of cases, increasing `hat_update_freq` beyond 1 causes the number of outer iterations to increase substantially, to the point where the reduction in per-iteration cost does not translate into a faster overall fit. Discovering an automatic rule that adapts the schedule to the current distance from convergence remains as future work.

Systematic benchmarking has been conducted primarily on binomial, Poisson, and Gaussian models. Performance on families with non-canonical dispersion structures, particularly inverse Gaussian, warrants further investigation. The codebase is designed to accommodate these families correctly, but their inclusion in the timing analysis is left as future work.

6 Conclusions and Future Work

This chapter restates the project aims, summarises the technical achievements, reflects on the project experience, and identifies directions for future development.

6.1 Summary of Aims

This project set out to improve the computational performance of `brglm2` by replacing quasi-Fisher scoring with a trust region framework, whilst preserving the statistical validity of all supported bias-reduction methods and maintaining strict backward compatibility. The work combined statistical knowledge of bias-reduced GLMs, numerical analysis of trust region and CG-Steihaug methods, and software engineering discipline for open-source contribution.

6.2 Achievements

The principal technical contributions are: the `compute_fit` helper function, which modularises the core fitting logic and enables lazy evaluation of expensive operations via Boolean flags; the trust region outer loop with adaptive radius control; the preconditioned CG-Steihaug subproblem solver with Jacobi preconditioning and Eisenstat-Walker adaptive tolerance; the Newton step-halving procedure for dispersion with analytical geometry injection; the adaptive hat value scheduling strategy that reduces QR decomposition cost across iterations; the vectorised `AS_median` adjustment that reduces per-iteration cost from $\mathcal{O}(np^3)$ to $\mathcal{O}(np^2)$ and is the primary source of the large high-dimensional speedups; and automatic sparse matrix dispatch for high-dimensional covariate structures via Cholesky-based hat value computation and slot-level row scaling. Of 394 existing tests, 392 pass; the remaining two are understood and addressed. Backward compatibility is preserved throughout, and the implementation is structured for upstream contribution to `brglm2`.

6.3 Discussion

The shift from gradient descent to trust region optimisation, proved both mathematically demanding and ultimately more principled than the original specification anticipated. Adapting general trust region theory (Nocedal and Wright 2006) to bias-reduced GLMs required careful analysis which extended the first-term timeline and had knock on effects to the timetable for term 2.

Several important lessons emerged. The profiling-first development methodology proved indispensable allowing development and research to be directed at the biggest time saving approaches. The investment in comprehensive testing and benchmarking infrastructure, though time-consuming to establish, provided objective evidence of correctness and performance throughout development and ensured that algorithmic changes did not inadvertently alter statistical results.

It is also worth noting that the largest performance gains arose not from the trust region framework alone but from the combination of the trust region with the vectorised `AS_median` adjustment. Tables 4 and 5 confirm this directly: the trust region with the looped adjustment is consistently two to three times slower than quasi-Fisher scoring across mid-range dense scenarios, while the vectorised adjustment with the trust region achieves $31.65\times$ at $n = 1000$, $p = 200$.

6.4 Future Work

Several natural extensions follow from this work.

First, ensuring that all tests pass is a necessary next step for the work to ensure it can be published. As such getting the correction type working is a suitable first place for future time to be focused.

A hybrid algorithm that adaptively selects between the trust region and quasi-Fisher scoring based on estimated problem cost is the highest-priority extension. Comparing np^2 against a configurable threshold would allow the existing default behaviour to be preserved for small problems while engaging the trust region solver for large ones, and is the most likely prerequisite for upstream integration into `brglm2`. Either this or an interface system that allows the user to pick with detailed documentation and recommendations on which one to use for their given problem.

A more principled hat value update schedule, driven by a convergence criterion on the change in h_i across iterations rather than a fixed frequency, would replace the current user-controlled parameter with an automatic rule. This would address the observed cases where increasing `hat_update_freq` increases iteration count sufficiently to negate the per-iteration saving.

Extension of systematic timing to the inverse Gaussian family would close the remaining gaps in benchmark

coverage. A simulation study confirming nominal confidence interval coverage across all families would complete the statistical validation.

Exploring other preconditioning methods, including an incomplete Cholesky factorisation $M \approx \mathbf{L}\mathbf{L}^T$, to see whether the more expensive application is worthwhile trade off for further reducing the number of CG iterations.

Rcpp integration (Eddelbuettel and Balamuta 2018) for the inner matrix-vector products in the CG-Steihaug loop, or more generally, would provide further performance gains without changing the algorithm’s structure.

6.5 Closing Statement

By replacing the QR-dominated quasi-Fisher scoring loop with a trust region framework and vectorised adjustment computations, this project demonstrates that principled bias-reduced inference at high dimensionality is computationally feasible. The results remove a barrier that has until now made `brglm2` impractical in precisely the large-scale settings where its statistical guarantees matter most.

7 Legal, Social, Ethical and Professional Considerations

7.1 Ethical Considerations

This project does not involve human participants, collection of personal data, or interaction with individuals as research subjects. As such, no ethical approval is required, and no application has been submitted to the University’s Research Ethics Committee.

7.2 Open Source Licensing

The `brglm2` package is distributed under the GPL-3 licence, which governs how the software can be used, modified, and redistributed. All code contributed as part of this project will be released under the same licence, ensuring the work remains freely available to the research and data analysis community. All contributions will be appropriately attributed and will follow the package’s existing contribution guidelines as documented in the repository.

7.3 Professional Considerations

As this project involves contributing to a widely-used software package, professional software development standards are paramount. Code quality, readability, and maintainability are requirements, with all contributions being reviewed by Prof. Kosmidis before integration. Backward compatibility is maintained so that user-facing interfaces remain stable and the implementation does not alter existing analyses. Any assertions about performance improvements are substantiated by reproducible benchmarks, and no exaggerated claims about speedups are made.

7.4 Social Impact

Improving the computational performance of `brglm2` has positive social implications by making rigorous statistical methods accessible to researchers working with large datasets. Bias-reduced inference has important applications in medical research, social science, epidemiology, and ecology, fields where finite-sample correctness is not merely a statistical nicety but a practical necessity. By reducing computational barriers, this work potentially enables more widespread use of these principled statistical approaches in settings where the scale of data has previously made them infeasible.

References

- Agresti, Alan. 2015. *Foundations of Linear and Generalized Linear Models*. John Wiley & Sons.
- Bates, Douglas, Martin Maechler, and Mikael Jagan. 2024. *Matrix: Sparse and Dense Matrix Classes and Methods*. <https://CRAN.R-project.org/package=Matrix>.
- Chang, Winston, and Javier Luraschi. 2023. *Profvis: Interactive Visualizations for Profiling r Code*. <https://CRAN.R-project.org/package=profvis>.
- Cordeiro, Gauss M, and Peter McCullagh. 1991. “Bias Correction in Generalized Linear Models.” *Journal of the Royal Statistical Society: Series B (Methodological)* 53 (3): 629–43.
- Eddelbuettel, Dirk, and James Joseph Balamuta. 2018. “Extending R with C++: A Brief Introduction to Rcpp.” *The American Statistician* 72 (1): 28–36.
- Eisenstat, Stanley C, and Homer F Walker. 1996a. “Choosing the Forcing Terms in an Inexact Newton Method.” *SIAM Journal on Scientific Computing* 17 (1): 16–32.
- . 1996b. “Choosing the Forcing Terms in an Inexact Newton Method.” CRPC-TR94463. Center for Research in Parallel Computation, Rice University. <https://softlib.rice.edu/pub/CRPC-TRs/reports/CRPC-TR94463.pdf>.
- Firth, David. 1993. “Bias Reduction of Maximum Likelihood Estimates.” *Biometrika* 80 (1): 27–38.
- Hester, Jim, and Davis Vaughan. 2025. *Bench: High Precision Timing of r Expressions*. <https://CRAN.R-project.org/package=bench>.
- Hutchinson, Michael F. 1990. “A Stochastic Estimator of the Trace of the Influence Matrix for Laplacian Smoothing Splines.” *Communications in Statistics – Simulation and Computation* 19 (2): 433–50.
- Kenne Pagui, Euloge Clovis, Alessandra Salvan, and Nicola Sartori. 2017. “Median Bias Reduction of Maximum Likelihood Estimates.” *Biometrika* 104 (4): 923–38.
- Kingma, Diederik P, and Jimmy Ba. 2015. “Adam: A Method for Stochastic Optimization.” In *Proceedings of the 3rd International Conference on Learning Representations (ICLR)*.
- Kosmidis, Ioannis. 2023. *Brglm2: Bias Reduction in Generalized Linear Models*. <https://CRAN.R-project.org/package=brglm2>.
- Kosmidis, Ioannis, and David Firth. 2009. “Bias Reduction in Exponential Family Nonlinear Models.” *Biometrika* 96 (4): 793–804.
- . 2021. “Jeffreys-Prior Penalty, Finiteness and Shrinkage in Binomial-Response Generalized Linear Models.” *Biometrika* 108 (1): 71–82.
- Kosmidis, Ioannis, Euloge Clovis Kenne Pagui, and Nicola Sartori. 2020. “Mean and Median Bias Reduction in Generalized Linear Models.” *Statistics and Computing* 30: 43–59.
- Mersmann, Olaf. 2023. *Microbenchmark: Accurate Timing Functions*. <https://CRAN.R-project.org/package=microbenchmark>.
- More, Jorge J, and Danny C Sorensen. 1983. “Computing a Trust Region Step.” *SIAM Journal on Scientific and Statistical Computing* 4 (3): 553–72.
- Nocedal, Jorge, and Stephen J Wright. 2006. *Numerical Optimization*. 2nd ed. Springer Series in Operations Research and Financial Engineering. Springer.
- Steihaug, Trond. 1983. “The Conjugate Gradient Method and Trust Regions in Large Scale Optimization.” *SIAM Journal on Numerical Analysis* 20 (3): 626–37.
- van der Loo, Mark. 2020. “Comparing R Objects with `tinytest`.” *The R Journal* 12 (2): 164–77. <https://doi.org/10.32614/RJ-2020-050>.
- Wickham, Hadley, Jim Hester, Winston Chang, and Jennifer Bryan. 2023. *Devtools: Tools to Make Developing r Packages Easier*. <https://CRAN.R-project.org/package=devtools>.
-

Acknowledgements

I would like to gratefully acknowledge the supervision and guidance of Prof. Ioannis Kosmidis throughout this project. His deep expertise in both the statistical foundations of bias-reduced estimation and the inner workings of the `brglm2` codebase was invaluable at every stage, and his patience in working through the subtleties of the trust region formulation and the adjusted score objective was certainly not unnoticed.

I would also like to thank my second assessor, Dr. Fanghui Liu, for his encouraging and constructive feedback on the Progress Report.

None of this would have been possible without the unwavering support of my family. My parents have been a constant source of encouragement, not just through this degree but throughout everything that led to it — pushing me to be the best version of myself long before I arrived at university, and never doubting that I would get here. My extended family have been just as generous with their support, and more than a few of my granny’s biscuits can legitimately be credited with getting me through late-night sessions. I also want to remember my great-auntie Carol, who we lost just before I started university. As the first in my immediate family to reach this point, I know she would have loved to see it, and I hope she would have been proud.

Finally, I want to acknowledge my partner Megan. Her belief in me is often greater than my own, and her patience, encouragement, and willingness to listen to me natter means more to me than I can easily put into words.

Appendix A: Profiling Output

Profiling was conducted using `profvis` (Chang and Luraschi 2023) on a representative model fit (MultipleFeatures, $n = 2000$, $p = 140$). Flame graphs from the full `brglmFit` workflow and the isolated `compute_fit` helper are provided below.

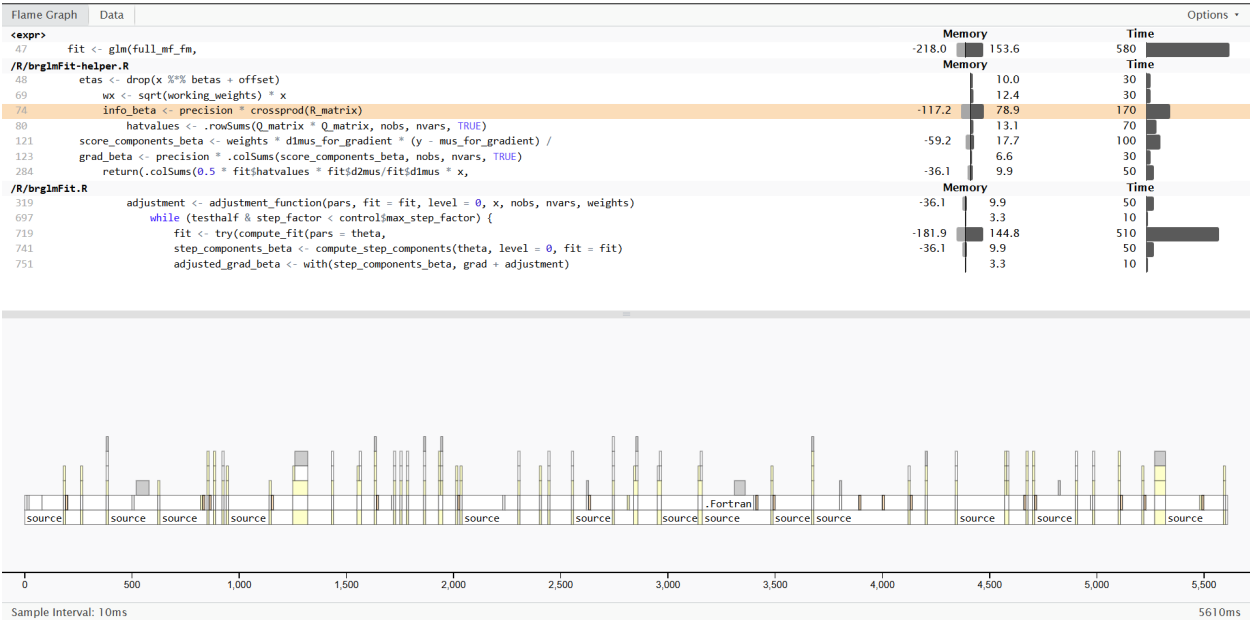


Figure 1: Main Profiling



Figure 2: Fit Profiling

Appendix B: Benchmark Tables

Tables B.1-B.3 presents the test suite execution time baseline, confirming that the `compute_fit` abstraction introduces no overhead relative to the original implementation.

Table B.1 — Test Suite Execution Time

Version	Total runtime (s)
Original	35.9
Refactored	37.3
Speedup	0.96×

Table B.2 — Microbenchmark Results on Standard Datasets (*execution time in milliseconds, 30 replications*)

Dataset / Model	Version	Min	Median	Max	Speedup
Lizards (1000 evals)	Original	4.17	4.64	23.89	—
Lizards (1000 evals)	Refactored	4.12	4.64	20.39	1.001×

Dataset / Model	Version	Min	Median	Max	Speedup
Endometrial Probit (500 evals)	Original	5.45	6.05	174.71	—
Endometrial Probit (500 evals)	Refactored	5.44	6.00	14.95	1.008×

Table B.3 — High-Dimensional Benchmarks (*execution time in seconds*)

Scenario	Original	Refactored	Speedup
MultipleFeatures ($n = 2000$, $p = 140$, $\kappa = 0.14$)	8.607	8.566	1.005×
MDYPL MultipleFeatures ($\alpha = 1/(1 + \kappa)$)	0.1088	0.1076	1.011×

Table B.4 — Standard Package Datasets: AS_mixed (*bench::mark with 150 interleaved iterations for MultipleFeatures; microbenchmark with 5000/2500 replications for Lizards/Endometrial; results volatile across runs*)

Dataset	n	p	Original (median)	New (median)	Speedup range
Lizards	24	5	—	—	0.68×–1.03×
Endometrial probit	79	4	—	—	0.68×–1.03×
MultipleFeatures (brglmFit)	1000	140	2.85 GB alloc	1.95 GB alloc	0.94×–1.38×
MultipleFeatures (mdyplFit)	1000	140	—	—	≈ 1.0×

Note: MultipleFeatures memory figures are total allocations across the fit. Timing ranges reflect six benchmark runs conducted between 24 March and 2 April 2026.

Table B.5 — Coefficient matching over all synthetic scenarios

Scenario	n	p	$\max \Delta\hat{\beta} $ (AS_mixed)	$\max \Delta\hat{\beta} $ (AS_median)
Logistic AR-1	200	5	2.42×10^{-8}	5.91×10^{-7}
Logistic AR-1	1000	20	1.87×10^{-7}	2.08×10^{-7}

Scenario	n	p	$\max \Delta\hat{\beta} $ (AS_mixed)	$\max \Delta\hat{\beta} $ (AS_median)
Logistic AR-1	4000	50	6.14×10^{-9}	6.05×10^{-9}
Logistic AR-1	8000	100	8.22×10^{-8}	8.09×10^{-8}
Probit block-corr	500	10	6.55×10^{-7}	6.89×10^{-7}
Probit block-corr	2000	30	1.85×10^{-7}	1.89×10^{-7}
Probit block-corr	5000	60	3.64×10^{-8}	3.72×10^{-8}
Poisson sparse	500	10	6.86×10^{-7}	5.89×10^{-7}
Poisson sparse	2000	25	8.63×10^{-8}	8.20×10^{-8}
Logistic high-dim	500	100	8.93×10^{-7}	9.67×10^{-7}
Logistic high-dim	1000	200	3.08×10^{-7}	3.13×10^{-7}
Cloglog survival	800	15	1.09×10^{-7}	2.65×10^{-7}
Cloglog survival	3000	40	4.30×10^{-7}	4.48×10^{-7}
One-hot (10 $\times$ 20 levels)	2000	191	6.32×10^{-7}	5.58×10^{-7}
One-hot (20 $\times$ 50 levels)	5000	981	5.75×10^{-7}	—
One-hot (5 $\times$ 100 levels)	10000	496	1.28×10^{-7}	1.35×10^{-7}
Interaction dummies ($n_{\text{base}} = 15$)	2000	120	3.27×10^{-7}	5.35×10^{-7}
Interaction dummies ($n_{\text{base}} = 20$)	5000	210	5.07×10^{-7}	8.88×10^{-8}
Block diagonal (10 groups, $p = 8$)	5000	80	1.19×10^{-7}	6.93×10^{-8}
Block diagonal (20 groups, $p = 10$)	20000	200	1.93×10^{-8}	2.10×10^{-8}